THE KERNEL BORDERLANDS

FIELD MANUAL TO APPROACH A KERNEL SECURITY SOFTWARE SYSTEM

ARCH LINUX EDITION

PACKAGING ▪ SERVICES ▪ COMMANDS ▪ CONFIGURATION

Package: `kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst`

Release 0.1.0 ▪ Covers kbd, kbd_sensor, kbctl, kbopd, mcp-server, kb-checker, kb-aads

CONTENTS

I Orientation 7

1 What You Are Operating 9

1.1 The one-paragraph version 9

1.2 The five subsystems 9

1.3 The data flow, end to end 10

1.4 The four operator surfaces 10

1.5 Design invariants you must not break 11

1.6 What is verified, and what is not 11

2 Ground Truth: Sockets, Ports, Paths, Users 13

2.1 Sockets under `/run/kb/` 13

2.2 Network ports 14

2.3 Filesystem layout after install 14

2.4 Users, groups, and permissions 15

2.5 Environment variables 15

3 Before You Install: Kernel Prerequisites 17

3.1 The checklist 17

3.2 Enabling BPF LSM when it is missing 18

3.3 What runs where 18

II Packaging 19

4 Anatomy of the Package 21

4.1 From source to artifact 21

4.2 The five build stages 21

4.3 The Makefile contract 22

4.4 What the unit files are rewritten to . . . 23
4.5 Package metadata and maintainer scripts . . . 24
4.6 What is deliberately not in the package . . . 24

5 Building the Package on Arch Linux 27
5.1 Prerequisites . . . 27
5.2 Step by step . . . 28
5.2.1 Why the PKGBUILD is unusual . . . 28
5.3 Inspect the artifact before you trust it . . . 29
5.4 Cross-distro note . . . 30

6 Installing, Verifying, Upgrading, Removing 31
6.1 Install . . . 31
6.1.1 What the post-install step did . . . 31
6.2 Automatic repair of a legacy dev install . . . 32
6.3 Verify the installation . . . 32
6.4 Upgrade . . . 33
6.5 Remove . . . 33

III Operations 35

7 Starting the Stack 37
7.1 The boot order . . . 37
7.2 The one-command start . . . 38
7.3 The same thing with `systemctl` . . . 39
7.4 Watching a start succeed . . . 39
7.5 Stopping, restarting, and what to expect . . . 40

8 The Web Dashboard 41
8.1 How it is put together . . . 41
8.2 Commands . . . 41
8.3 Reaching it from your workstation . . . 42
8.4 What the API exposes . . . 42
8.5 The API token: how the action buttons are authorized . . . 42
8.6 Symptoms and causes . . . 43

9 The SSH Terminal Console 45
9.1 The architecture . . . 45
9.2 Commands . . . 45
9.2.1 `add-key` in detail . . . 46
9.3 The sshd configuration, line by line . . . 46
9.4 Check the console on your host . . . 47
9.5 Using the TUI . . . 48
9.5.1 The four tabs . . . 48
9.5.2 Containing a process . . . 48
9.5.3 The query console . . . 48
9.5.4 When `kbd` is unreachable . . . 49

10 kbctl Reference 51
10.1 Command map . . . 51
10.2 Global flag . . . 52
10.3 Exit codes . . . 52
10.4 Stack control . . . 52
10.4.1 kbctl up . . . 52
10.4.2 kbctl down . . . 52
10.4.3 kbctl status . . . 53
10.5 Web dashboard . . . 53
10.5.1 kbctl dashboard . . . 53
10.5.2 kbctl dashboard open . . . 54
10.5.3 kbctl dashboard status . . . 54
10.5.4 kbctl dashboard url . . . 54
10.6 SSH console and TUI . . . 55
10.6.1 kbctl ssh . . . 55
10.6.2 kbctl ssh connect . . . 55
10.6.3 kbctl ssh add-key . . . 55
10.6.4 kbctl tui . . . 56
10.7 Containment and classification . . . 56
10.7.1 kbctl process isolate . . . 56
10.7.2 kbctl zone override . . . 57
10.8 Policy, workloads, audit, stats . . . 57

10.8.1 kbctl policy reload . . . 57
10.8.2 kbctl workload keygen . . . 57
10.8.3 kbctl workload sign . . . 58
10.8.4 kbctl workload reload . . . 58
10.8.5 kbctl audit verify . . . 58
10.8.6 kbctl audit export . . . 59
10.8.7 kbctl stats . . . 59
10.9 The daemons' own flags . . . 59
10.9.1 kbd . . . 59
10.9.2 kb-checker (kept down; for reference) . . . 60

11 Command Combinations and Playbooks 61
11.1 Playbook 1: cold start to first dashboard . . . 61
11.2 Playbook 2: a remote operator takes over . . . 61
11.3 Playbook 3: incident, from detection to containment . . . 62
11.4 Playbook 4: change policy without restarting . . . 62
11.5 Playbook 5: protect a workload with signed CWP rules . . . 62
11.6 Playbook 6: prove the audit log is intact . . . 63
11.7 Playbook 7: scripting and health gates . . . 64
11.8 Playbook 8: a quick demo for an audience . . . 64
11.9 Combination cheat sheet . . . 65

12 kb-checker: Under Maintenance and Repairs 67
12.1 How it is kept dead, and how you start it . . . 67
12.2 What you lose while it is down . . . 68
12.3 Design invariants that still apply . . . 68

13 Demonstration Runbook: Install to Uninstall 69
13.1 Time budget and layout . . . 69
13.2 Phase 0: Before you start . . . 70
13.3 Phase 1: Install . . . 70
13.4 Phase 2: Authorize an operator and start the stack . . . 71
13.5 Phase 3: Show the sensor and the control plane working . . . 72
13.6 Phase 4: Web dashboard . . . 73
13.7 Phase 5: The SSH console . . . 73

13.8 Phase 6: Contain a process . . . 74
13.9 Phase 7: The audit proof . . . 74
13.10 Phase 8: Change policy live . . . 75
13.11 Phase 9: Failure and recovery . . . 75
13.12 Phase 10: Optional, the agent swarm . . . 75
13.13 Phase 11: Stop everything . . . 76
13.14 Phase 12: Uninstall . . . 76
13.15 Phase 13: Reset between demonstrations . . . 77
13.16 Running install and checks automatically . . . 77
13.17 The whole demonstration on one page . . . 77

IV Configuration 79

14 Configuration Reference 81
14.1 Which file does what, and when it takes effect . . . 81
14.2 policy.yaml . . . 82
14.3 rules.yaml: the behavior engine . . . 83
14.4 workloads.yaml: Critical Workload Protection . . . 88
14.5 agents.yaml . . . 89
14.6 allowlist.yaml and kb.yaml . . . 90
14.7 Overriding without editing packaged files: systemd drop-ins . . . 91

15 kb-aads: The Agent Swarm (Optional) 93
15.1 Create the environment . . . 93
15.2 Run it . . . 94
15.3 How `kbd` finds it . . . 94
15.4 Hardening in the unit . . . 95

V Reference 97

16 Troubleshooting 99
16.1 First five commands, always . . . 99
16.2 Symptom table . . . 101
16.3 Build-time failures . . . 102

16.4 Where the logs are . . . 102
16.5 Documentation and implementation disagreements . . . 102

17 Acceptance Checklist 105
17.1 A. Pre-flight . . . 105
17.2 B. Install . . . 106
17.3 C. Start and the sensor . . . 106
17.4 D. Dashboard and API . . . 107
17.5 E. SSH console . . . 107
17.6 F. Containment and audit . . . 108
17.7 G. Maintenance hold . . . 108
17.8 H. Reboot and removal . . . 108

A Wire and Event Contracts 109
A.1 C to Go: `kbd.sock`, hand-packed structs . . . 109
A.2 Go to clients: `kba.sock`, `pb.KBEvent` . . . 109

B Kernel Hook Points 111

C Condensed Cross-Distribution Recipe 113
C.1 Build the Arch package . . . 113
C.2 Build the Ubuntu package . . . 113
C.3 Differences in one table . . . 114

D Complete systemd Unit Files 115
D.1 kbd.service . . . 115
D.2 kb-sensor.service . . . 116
D.3 kb-checker.service (held down) . . . 117
D.4 kbopd.service . . . 118
D.5 sshd@kb-operator.service . . . 119
D.6 kbagents.service . . . 120
D.7 kb-tui-session-wrapper.sh . . . 120
D.8 provision.sh . . . 121

E Glossary 123

F Package File Manifest 125

PART I

ORIENTATION

CHAPTER 1

WHAT YOU ARE OPERATING

1.1 The one-paragraph version

Kernel Borderlands (KB) is a kernel-level runtime defense framework for Linux. A set of eBPF sensors watch kernel telemetry at Ring 0; a Go control plane (`kbd`) ingests that telemetry, keeps per-process state, and enforces policy; a Python agent swarm (`kb-aads`) coordinates adaptive response; and four operator surfaces (`kbctl`, a terminal UI served over SSH, a web dashboard, and an MCP server) let humans and automation see and steer the system. A separate Rust watchdog (`kb-checker`) audits the rest of the stack.

This book is the field manual for *operating the packaged system*: how it is built into a `.pkg.tar.zst` package for Arch Linux, how that package is installed, how every service is started, what every command does, how the commands combine into working procedures, and what every configuration file means.

READ THIS FIRST: THE KB-CHECKER IS DOWN ON PURPOSE

`kb-checker`, the safety and integrity watchdog, is under maintenance and repairs and is kept down in this release. The package installs its binary and unit file, but nothing ever starts it for you: the package does not enable it (and removes any boot autostart), `kbctl up` skips it, and `kbctl status` prints `UNDER MAINTENANCE AND REPAIRS (kept down)`. It starts only if you start it by hand. Everything in this book is written for a stack *without* the watchdog. Chapter 12 explains what that means for you.

1.2 The five subsystems

KB is five subsystems kept in sync through shared wire and event contracts. A change in one often requires a matching change in another, which is why the package ships them together.

Directory	Language	Role and installed artifact
`kb-core/`	C (eBPF, CO-RE)	Ring-0 instrumentation and the userspace loader/bridge. Installed as `kbd_sensor`.
`kb-control-plane/`	Go	The `kbd` daemon: ingestion, storage, scoring, gRPC, HTTP/SSE, enforcement.
`kb-aads/`	Python	Agent Defense Swarm (multi-agent RL on Ray). Installed as sources plus the `kbagents` launcher.
`kb-checker/`	Rust	Stateless safety and integrity watchdog. Installed as `kb-checker` (***kept down***).
`kb-op/`	Go / Rust / TS	Operator interfaces: `kbctl`, `kbopd` (the TUI and embedded dashboard), `mcp-server`.

1.3 The data flow, end to end

Figure 1.1 is the picture to keep in your head for the rest of the book. Telemetry flows *up* from the kernel; containment commands flow *down* on a *separate* socket so a telemetry burst can never stall or kill delivery of a containment order.

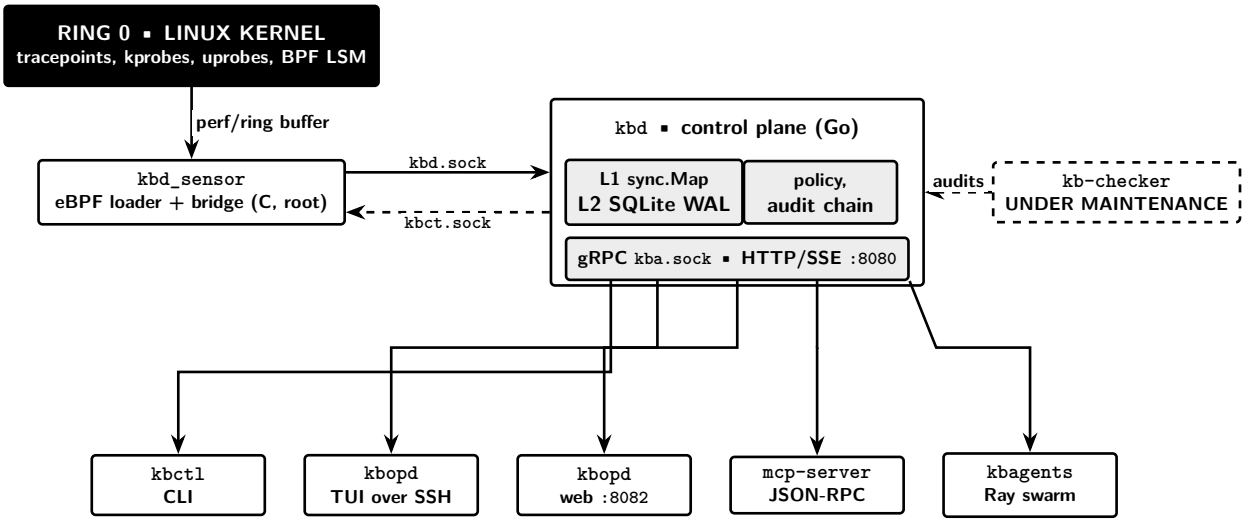


Figure 1.1: Data flow. Solid arrows carry telemetry and queries; the dashed arrow from `kbd` to `kbd_sensor` is the separate control socket that carries containment commands.

1.4 The four operator surfaces

Kernel-level operations demand availability and flexibility, so KB gives you four independent ways in. If the web port is firewalled you fall back to the SSH TUI; if the shell is restricted, automation scripts `kbctl`; if the system is run autonomously, an AI assistant uses the MCP gateway.

Surface	Transport	Use it for
`kbctl`	gRPC over `/run/kb/kba.sock`	Scripted playbooks, CI, rapid overrides, starting and stopping the other surfaces.
`kbopd` (TUI)	gRPC over UDS, served via the dedicated operator `sshd` on port 2222	Low-bandwidth triage on a bastion, no browser needed.
`kbopd` (web)	Static app on `127.0.0.1:8082`; REST + SSE to `kbd` on `127.0.0.1:8080`	Visual, live threat monitoring.
`mcp-server`	JSON-RPC 2.0 over stdio	AI agents querying state and recommending containment.

ONE BINARY, TWO FACES

`kbopd` is the compiled `kb-tui` binary installed under another name. Run with a terminal it draws the TUI; run without one (as the `kbopd.service` unit does) it serves the embedded web dashboard and starts a background `mcp-server`. That is why `kbctl tui` and `kbctl dashboard` both end up executing `kbopd`.

1.5 Design invariants you must not break

- Shared structs stay byte-identical. Wire structs are packed little-endian on both the C and Go sides (Appendix A). They change together or not at all.
- Rules and policy are compiled, not hand-edited on the sensor. `policy.yaml` and `rules.yaml` are read by `kbd` and *pushed* to the sensor when it connects.
- `kb-checker` is stateless and has no network listeners. This is a security invariant of the watchdog, not a style preference.
- Documented contracts beat code. Where this book and the repository documentation under `docs/` disagree, the documentation is canonical and the disagreement is flagged in Chapter 16.

1.6 What is verified, and what is not

A field manual that hides its own uncertainty is dangerous. This release was built and tested without root access on the build machines, so the status of every claim in this book is one of the following.

Claim	Status	Evidence
Both packages build, contain no key material	Verified	File lists inspected; `find` for `ssh_host*`, `authorized_keys`, `id_*` returns nothing.
Ubuntu binaries resolve all libraries on 24.04	Verified	`ldd` on every binary, no `not found`.
Unit files rewritten to `/usr/bin`	Verified	`grep ExecStart` on the extracted package.
`kbctl dashboard up/open/down`	Verified	Driven against the packaged `kbopd` through a stub `systemctl`; HTTP 200 from the embedded app.
SSH TUI path (wrapper, PTY, tabs, quit)	Verified	Unprivileged `sshd` with the packaged wrapper; exit status 0.
Offline banner when `kbd` is down	Verified	`[OFFLINE]` and `[DEMO DATA]` rendered in the TUI.
`kbagents` launcher and `kb-aads` swarm (Ubuntu)	Verified	Ran on the VM with its venv: status server up, nine agents active, 0 errors, checkpoint loaded. Not run under `systemd`.
`apt install` of the `.deb` on the Ubuntu 24.04 VM	Verified	Installed over a live dev install: user/shell, token file (640 `root:kb`), legacy conf and units migrated, `kb-checker` left stopped and disabled. `pacman -U` on Arch: not tested.
Units under real `systemd`, eBPF sensor under the package (Ubuntu VM)	Verified	`kbctl up`: `kbd`, sensor, `kbopd`, operator `sshd` active; sensor logging events, behavior-engine and zone transitions recorded, audit chain intact (1701 entries). Reboot persistence: not tested.
`kbd` HTTP, CORS and API token on the VM	Verified	Dashboard origin allowed, foreign origin not; `policy/reload` gave 401 (no token), 401 (wrong), 200 (right).
`operator` account with a `/bin/sh` shell and `ForceCommand` (replaces the `nologin` default)	Verified	On the Ubuntu VM: publickey accepted, TUI rendered with a live link to `kbd` (no `[OFFLINE]`), session audit callback reached `kbd`, clean exit. Three bugs were found and fixed on the way (sshd capabilities, locked account, `kba.sock` group). Final run: 34 of 34 checks passed. See Chapter 9.
Dashboard action buttons with the runtime token	Not tested in a browser	Build, served bundle, fragment parsing and `kbctl` hand-off tested; see Chapter 8.

Chapter 17 gives you the exact checklist to close every Not tested row on your own host, and Chapter 13 is the complete demonstration from install to uninstall.

CHAPTER 2

GROUND TRUTH: SOCKETS, PORTS, PATHS, USERS

Everything in later chapters refers back to this chapter. Memorize the shape, look up the details.

2.1 Sockets under `/run/kb/`

`/run/kb/` is created by `kbd.service` (`RuntimeDirectory=kb`, mode 0775) and is removed by systemd when `kbd` stops.

Socket	Direction	Protocol	Purpose
`kbd.sock`	`kbd_sensor` → `kbd`	packed structs	Telemetry only, one direction.
`kbct.sock`	`kbd` → `kbd_sensor`	packed structs	Containment and sensitive-path pushes. Split from `kbd.sock` so a telemetry burst cannot stall containment.
`kba.sock`	clients → `kbd`	gRPC, `root:kb` 0660	Enforcement IPC. Group `kb` so the `operator` SSH account and `kb` members can use it (`kbd.service` runs `chgrp kb` after start); this is full control-plane access. Used by `kbctl`, `kbopd`, `mcp-server`, `kb-aads`.
`kbc.sock`	`kb-checker` diagnostics	gRPC	Watchdog diagnostics (unused while the checker is down).

2.2 Network ports

Port	Owner	Bind	Notes
8080	`kbd` HTTP API + SSE	127.0.0.1 (flag `--http-addr`)	REST for processes, alerts, logs, services, metrics, agents, policy; SSE on `/api/events`. CORS allowlist from `KB_HTTP_CORS_ORIGIN`.
8082	`kbopd` embedded dashboard	127.0.0.1, fixed in the binary	Static Vite app plus `POST /api/containment`.
2222	operator `sshd`	all interfaces (`sshd@kb-operator`)	Key-only console that forces the TUI wrapper.
8601	`kb-aads` status server	127.0.0.1 (`KB_AADS_STATUS_HOST/PORT`)	Agent status; `kbd` proxies it for `/api/agents`.

PORT 2222 IS THE ONLY NON-LOOPBACK LISTENER

The dashboard and API bind loopback on purpose. Remote humans reach the system through SSH on 2222 (Chapter 9) rather than by exposing HTTP. Keep it that way unless you also set `KB_HTTP_API_TOKEN` and a TLS-terminating proxy.

2.3 Filesystem layout after install

```
/
├── usr/bin/                    kbd  kbd_sensor  kbctl  kbopd  mcp-server  kb-
│                               checker  kbagents  kb-tui-session-wrapper.sh
├── usr/lib/kb/                 provision.sh  aads/ (kb-aads sources)
├── usr/lib/systemd/system/     kbd kb-sensor kbopd kbagents kb-checker sshd@kb-
│                               operator (.service)
├── usr/share/doc/kernel-borderlands/LICENSE  README
├── etc/kb/                     policy.yaml rules.yaml workloads.yaml agents.yaml sshd_kb-
│                               operator.conf
│   ├── ssh/ssh_host_ed25519_key      created by provision.sh
│   ├── authorized_keys               created empty by provision.sh
│   └── api.env                       dashboard API token, root:kb 0640
├── var/lib/kbd/state.db        SQLite WAL, made by StateDirectory=kbd
├── run/kb/                     kbd.sock  kbct.sock  kba.sock  kbc.sock  (runtime)
└── opt/kb/venv/                optional, you create it for kb-aads
```

Figure 2.1: Where everything lands. Paths are identical on both distributions.

2.4 Users, groups, and permissions

provision.sh runs from the package's post-install step and is idempotent.

Object	Definition
group kb	System group; owns /etc/kb (mode 0750, root:kb).
user operator	System user, shell /bin/sh (a nologin shell refuses sshd's ForceCommand), home /, shadow password * (no usable password, but not locked: OpenSSH without PAM refuses a locked account even for key logins). Member of kb. The only account the operator sshd accepts (Match User operator).
/etc/kb/ssh/ssh_host_ed25519_key	Host key for the operator sshd, generated on first install, never overwritten.
/etc/kb/authorized_keys	Empty at install, operator:kb, mode 0600. Fill it with kbctl ssh add-key.
Config files in /etc/kb	Mode 0640 root-owned; kbd runs as root. Listed as package conffiles so your edits survive upgrades.

2.5 Environment variables

Variable	Read by	Meaning
KB_HTTP_CORS_ORIGIN	kbd	Comma-separated browser origins allowed to call the API. The packaged unit sets http://127.0.0.1:8082,http://localhost:8082. Default without it: the Vite dev origins on 5173 only.
KB_HTTP_API_TOKEN	kbd	Bearer token gating the mutating routes (isolate, restore, policy/reload). The package generates it into /etc/kb/api.env and kbd.service loads it. Unset means those routes fail closed.
KB_AADS_STATUS_HOST, KB_AADS_STATUS_PORT	kbagents	Bind address of the agent status server (default 127.0.0.1:8601).
KB_AADS_API_ADDR	kbd	Where kbd proxies /api/agents; must match the status server.

CHAPTER 3

BEFORE YOU INSTALL: KERNEL PREREQUISITES

The package can be installed on any Arch Linux host. The *sensor* will only do its full job on a kernel that exposes BTF and has the BPF Linux Security Module enabled. Check this first; it takes a minute and saves an hour.

3.1 The checklist

1 Confirm BTF is available

CO-RE (compile once, run everywhere) relies on kernel type information.

```
$ ls -l /sys/kernel/btf/vmlinux
-r--r--r-- 1 root root 5321944 ... /sys/kernel/btf/vmlinux
```

If the file is missing, the sensor cannot relocate its programs on this kernel.

2 Confirm the kernel was built with BPF LSM

```
$ zgrep CONFIG_BPF_LSM /proc/config.gz  # or: grep CONFIG_BPF_LSM /boot/config-$(uname -r)
CONFIG_BPF_LSM=y
```

3 Check whether `bpf` is in the active LSM list

```
$ cat /sys/kernel/security/lsm
lockdown,capability,landlock,yama,apparmor,bpf
```

The trailing `bpf` means the in-kernel file-access block (`kb_lsm_file_open`) can attach. Without it, the sensor still loads its tracing programs; only the LSM program fails to attach, and the sensor's startup log says so rather than silently doing nothing.

3.2 Enabling BPF LSM when it is missing

Add `bpf` to the kernel's `lsm=` boot parameter, regenerate the boot configuration, reboot.

Arch Linux (GRUB)

```
$ sudo $EDITOR /etc/default/grub
  GRUB_CMDLINE_LINUX_DEFAULT="quiet lsm=landlock,lockdown,yama,apparmor,bpf"
$ sudo grub-mkconfig -o /boot/grub/grub.cfg
$ sudo reboot
```

SYSTEMD-BOOT / OTHER LOADERS

If you use `systemd-boot`, append the same `lsm=...` string to the `options` line of your entry under `/boot/loader/entries/`. Arch's own kernels build BPF LSM in; you only need the parameter. Keep `apparmor` in the list only if you actually run it; listing a module your kernel lacks is harmless but pointless.

4 Verify after reboot

```
$ cat /sys/kernel/security/lsm
```

`bpf` must appear at the end of the list.

3.3 What runs where

Component	Privilege	Why
`kbd_sensor`	root	Loads eBPF, attaches LSM and uprobes; needs CAP_BPF, CAP_PERFMON, CAP_SYS_ADMIN (exact set is kernel-dependent).
`kbd`	root	Owns `/run/kb`, `/var/lib/kbd`, binds the sockets.
`kbopd` (service)	unprivileged-style hardening	Only a loopback listener and a child process.
`sshd@kb-operator`	root, then drops to `operator`	Standard `sshd` privilege separation.

A VM OR CONTAINER SHARES THE HOST KERNEL ASSUMPTIONS

Inside a container the sensor sees the *host* kernel. Inside a VM it sees the *guest* kernel, which is what you want for the demo. Whichever you use, run the checklist above *inside* it.

PART II

PACKAGING

CHAPTER 4

ANATOMY OF THE PACKAGE

One Makefile target, `make install-pkg`, defines what ships. Both the `.deb` and the Arch package are thin wrappers around it, so the two distributions cannot drift apart.

4.1 From source to artifact

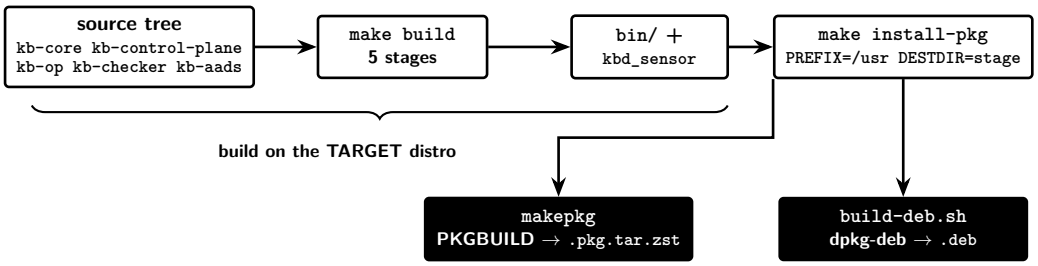


Figure 4.1: The packaging pipeline. The brace marks the part that must run on the distribution you are targeting, because the binaries link the build host's C library.

BUILD ON THE DISTRIBUTION YOU SHIP TO

The Go, Rust and C binaries are dynamically linked against the *build host's* glibc. The Arch host that builds the Arch package has glibc 2.44; Ubuntu 24.04 has 2.39. A binary built on the former will not start on the latter ("version `GLIBC_2.4x` not found"). That is why the `.deb` is built on the Ubuntu machine and the Arch package on Arch. Building on the *older* distribution always works forward; the reverse does not.

4.2 The five build stages

`make build` runs these in order and puts every product into the top-level `bin/` (the sensor stays in `kb-core/build/`).

Stage	Output	What happens
1/5	kb-core/build/kbd_sensor	clang compiles the BPF programs and userspace loader, bpftool generates the CO-RE skeletons, the vendored libbpf/src/libbpf.a is linked statically. Hardened with -D_FORTIFY_SOURCE=3, stack protector, -fPIE, full RELRO.
2/5	bin/kbd	go build -a of kb-control-plane/cmd/kbd.
3/5	bin/kb-checker	cargo build --release of the watchdog (built even though it is kept down).
4/5	bin/kbctl	go build -a of the CLI.
5/5	bin/mcp-server, bin/kbopd	Builds the MCP server; runs npm ci (only if node_modules is absent) and npm run build for the dashboard; then cargo build --release of kb-tui, which *embeds* the dashboard's dist/ at compile time and is copied to bin/kbopd.

ORDER MATTERS IN STAGE 5

The dashboard must be built *before* cargo compiles kb-tui. The Rust code embeds kb-dashboard/dist/ with rust-embed; if the folder is missing the compile fails with "folder ... does not exist". The Makefile already enforces the order.

4.3 The Makefile contract

Variable	Default	Meaning
PREFIX	/usr/local	Packages pass /usr.
BINDIR	$(PREFIX)/bin	Where binaries land; also substituted into unit files and the SSH wrapper.
LIBDIR	$(PREFIX)/lib	kb/provision.sh and kb/aads/.
SYSCONFDIR	/etc	Config root (/etc/kb).
UNITDIR	/etc/systemd/system for /usr/local, else $(PREFIX)/lib/systemd/system	Where unit files go.
DESTDIR	empty	Staging root, prepended to every install path.
GOARCH	go env GOARCH	Target architecture of the Go binaries.

Target	Effect
make build	The five stages above.
make install	Installs the six binaries to $(DESTDIR)$(BINDIR). Historical behaviour: sudo make install still goes to /usr/local/bin.
make install-pkg	install plus the wrapper, kbagents, units (path-rewritten), operator sshd config, default /etc/kb/*.yaml, provision.sh, the kb-aads sources, and docs.
make clean	Removes build products.

THE PACKAGE USES EXPLICIT FILE LISTS ON PURPOSE

The source tree contains development-only `ssh_host_ed25519_key` and `authorized_keys` files under `kb-core/` and `kb-control-plane/`. They are git-ignored, but they exist on disk. `install-pkg` names every file it ships and never copies a directory wholesale (except the `kb-aads` Python packages, which are listed folder by folder). If you extend the package, keep it that way and re-run the secrets check in Section 5.3.

4.4 What the unit files are rewritten to

The sources in `packaging/systemd/` use the development path `/usr/local/bin`. During `install-pkg`, `sed` rewrites it to `$(BINDIR)` (`/usr/bin`) in every unit and in the SSH wrapper, and rewrites the operator `sshd` config path from `/etc/ssh/sshd_config.d/kb-operator.conf` to `/etc/kb/sshd_kb-operator.conf`.

WHY THE SSHD CONFIG MOVED

Ubuntu's main `sshd` reads `Include /etc/ssh/sshd_config.d/*.conf`. A file there containing `Port 2222` is therefore also loaded by the *host's own* `sshd`, which grabs port 2222 first; the dedicated `sshd@kb-operator` instance then dies with `Address already in use` and restarts forever. This was observed on the live Ubuntu VM. The packages avoid it by keeping the file under `/etc/kb/`, where only the dedicated unit reads it.

4.5 Package metadata and maintainer scripts

Item	Ubuntu .deb	Arch
Source of truth	packaging/deb/	packaging/arch/PKGBUILD
Runtime deps	libc6, systemd, openssh-server, libelf1t64, zlib1g	glibc systemd openssh libelf zlib
Optional	python3, python3-venv (Recommends)	python (optdepends)
Config preserved on upgrade	conffiles (five files)	backup=(...) array (five files)
After install	postinst runs provision.sh, migrate-legacy.sh, systemctl daemon-reload	.install post_install / post_upgrade do the same and print next steps
Before removal	prerm disables and stops the units	pre_remove does the same
After removal	postrm reloads systemd; keeps /etc/kb, state, users	post_remove reloads systemd; same retention
Build flags	plain make build	!lto, neutral CFLAGS/LDFLAGS/RUSTFLAGS

Nothing is enabled or started on install. kb-checker can legitimately run a network lockdown; a package must never arm that by surprise. You start the stack yourself with kbctl up (Chapter 7).

4.6 What is deliberately not in the package

- Development keys and datasets. No private key, no authorized_keys from the tree, no scripts/dataset, no zip bundles.
- The Python virtualenv. The kb-aads environment (Ray, PyTorch) is multiple gigabytes and CPU-specific; you create it on the target (Chapter 15).
- Build outputs of other subsystems such as target/ directories and node_modules.

The kb-aads source tree *does* include one trained policy checkpoint, marl/checkpoints/containment_ppo/ (16 .pkl files, 1.8 MB). It is shipped because the containment agent loads it at runtime (RLModule.from_checkpoint in agents/containment.py); without it the agent reports model_loaded: False and recommends no containment. Pickle files execute code when loaded, so:

- They ship root-owned and not world-writable, under /usr/lib/kb/aads/, which only root

may modify.

- The package carries an integrity manifest. Verify it any time, and before every start if you are cautious:

```
$ cd /usr/lib/kb/aads/marl/checkpoints && sha256sum -c SHA256SUMS
containment_ppo/learner_group/state.pkl: OK
...
```

- The unused `jury_ppo` checkpoint and the development `session.log` are *not* shipped.

The manifest protects against accidental damage and casual tampering; an attacker with root can replace both files, which is why `kb-checker`'s eventual file-integrity role matters (Chapter 12).

CHAPTER 5

BUILDING THE PACKAGE ON ARCH LINUX

This chapter takes you from a source tree to a verified `kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst`. Every command is shown with the result you should see. Run it on Arch Linux itself (Chapter 5).

5.1 Prerequisites

```
$ sudo pacman -S --needed base-devel go rust clang bpf libbpf \
      nodejs npm libelf zlib
```

Package	Provides	Why
`go`	**Go 1.25 or newer**	`kbd`, `kbctl`, `mcp-server`. `go.mod` **declares** `go 1.25.0`.
`rust`	`cargo`, `rustc`	`kb-checker`, `kb-tui` **(becomes** `kbopd`**).**
`clang`, `bpf`	**BPF compiler,** `bpftool`	**CO-RE skeleton generation for the sensor.**
`libbpf`	`/usr/include/bpf/libbpf.h`	**Headers the userspace loader includes (the archive itself is vendored).**
`nodejs`, `npm`	**Vite toolchain**	**Builds the embedded dashboard.**

CHECK YOUR PATH: ALTERNATE TOOLCHAINS WILL BREAK THE BUILD

If a Nix profile, `asdf`, `rustup` **or similar sits earlier in** `PATH`**, it can shadow the system tools. A Nix** `clang` **wrapper does not search** `/usr/include`**, so the sensor fails with** `fatal error: 'bpf/libbpf.h' file not found`**. Check with** `which -a clang go cargo` **and, if needed, build from a clean path:** `PATH=/usr/local/bin:/usr/bin make build`**. The** `PKGBUILD` **pins this for you.**

5.2 Step by step

1 Get the source tree onto the Arch machine

```
$ cd ~/Desktop/Dev/kbworkspaces/project/kernel-borderlands-vm
$ ls
CHANGELOG.md  Makefile  kb-aads  kb-checker  kb-control-plane  kb-core  kb-op  packaging ...
```

2 Build the package

```
$ cd packaging/arch
$ makepkg -f
==> Making package: kernel-borderlands 0.1.0-1
==> Starting build()...
[Stage-1/5] Build C-based low-level eBPF core library bindings
[Stage-2/5] Build the main Go system daemon (kbd) into the root bin/
...
==> Starting package()...
==> Creating package "kernel-borderlands"...
==> Finished making: kernel-borderlands 0.1.0-1
```

The first build compiles the Go standard library (`-a`) and every Rust dependency; expect several minutes. Later builds are incremental.

3 Locate and fingerprint the artifact

```
$ ls -l kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
-rw-r--r-- 1 you you 25211681 ... kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
$ sha256sum kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
```

5.2.1 Why the PKGBUILD is unusual

- **It builds from the *surrounding working tree*, not a tarball (`source=()` is empty and `_root()` resolves `$startdir/../..`). Run `makepkg` from `packaging/arch/`.**
- **`options=(!debug !strip !lto)`. Arch's default `-flto` turns the C objects of Rust's `ring` crate (used by `kb-checker`) into LTO bitcode that `rustc`'s linker cannot resolve; the symptom is a wall of `undefined symbol: ring_core_0_17_14_...`.**
- **It runs `unset CFLAGS CXXFLAGS CPPFLAGS LDFLAGS RUSTFLAGS CC CXX` before `make build`. Exported makepkg flags (`-march=native`, `--as-needed`, frame pointers) break `kb-checker`'s link against libbpf (`undefined symbol: bpf_prog_get_next_id`) and would make the package CPU-specific. The project applies its own hardening in `kb-core/Makefile`.**
- **The `makedepends` line lists `libbpf` because the sensor's loader includes its headers.**

5.3 Inspect the artifact before you trust it

Never install a package you have not looked inside. These four checks take seconds.

1 List the contents

```
$ pacman -Qlp kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst | grep -v '/aads/'
kernel-borderlands /etc/kb/agents.yaml
kernel-borderlands /etc/kb/policy.yaml
...
kernel-borderlands /usr/bin/kbctl
kernel-borderlands /usr/bin/kbd
kernel-borderlands /usr/lib/systemd/system/kbd.service
...
```

2 Read the metadata

```
$ pacman -Qip kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
Name            : kernel-borderlands
Version         : 0.1.0-1
Depends On      : glibc  systemd  openssh  libelf  zlib
Installed Size  : 61.10 MiB
```

3 Prove no key material is inside

```
$ pacman -Qlp kernel-borderlands-*.pkg.tar.zst \
    | grep -E 'ssh_host|authorized_keys|id_(rsa|ed25519)|\.pem' \
    || echo "none"
none
```

4 Check every binary resolves its libraries

```
$ mkdir /tmp/kbx && bsdtar -xf kernel-borderlands-*.pkg.tar.zst -C /tmp/kbx
```

```
$ for b in kbd kbctl kbopd kbd_sensor kb-checker mcp-server; do
>   ldd /tmp/kbx/usr/bin/$b | grep 'not found'
> done
$ echo "(no lines above = all libraries resolved)"
```

FIELD TIP

Keep the `sha256sum` of each release next to the artifact. When a colleague asks “is this the package you tested?”, the hash answers in one line.

5.4 Cross-distro note

To produce the Ubuntu package you must build on Ubuntu 24.04 (or older); see the Ubuntu edition of this manual or Appendix C for the condensed recipe.

CHAPTER 6

INSTALLING, VERIFYING, UPGRADING, REMOVING

6.1 Install

```
$ sudo pacman -U kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
resolving dependencies...
looking for conflicting packages...
Packages (1) kernel-borderlands-0.1.0-1
:: Proceed with installation? [Y/n]
(1/1) installing kernel-borderlands
group kb already exists, skipping      # or: created group: kb
created user: operator (member of kb)
generated SSH host key: /etc/kb/ssh/ssh_host_ed25519_key
created empty /etc/kb/authorized_keys - add operator public keys here
provisioning complete
==> Kernel Borderlands installed. Nothing has been started.
```

6.1.1 What the post-install step did

1. **Created group** `kb` **and system user** `operator` **(no home, shell** `/bin/sh`**) if absent.**
2. **Created** `/etc/kb` **(**`root:kb`**, 0750) and** `/etc/kb/ssh/`**.**
3. **Generated** `/etc/kb/ssh/ssh_host_ed25519_key` ***only if it did not exist*****. Re-installing never rotates it.**
4. **Created an empty** `/etc/kb/authorized_keys` **(**`operator:kb`**, 0600).**
5. **Ran** `systemctl daemon-reload` **so the new units are visible.**
6. **Ran** `/usr/lib/kb/migrate-legacy.sh` **(Section 6.2): repaired a legacy operator-sshd install if one exists, and removed any boot autostart of** `kb-checker`**.**

It did not enable or start anything (on a fresh install it also made sure `kb-checker` **is stopped).**

6.2 Automatic repair of a legacy dev install

If the host previously ran the development install (`install.sh` or `make install`), the package repairs it for you; there is nothing to do by hand. `/usr/lib/kb/migrate-legacy.sh` runs from the post-install step, is idempotent, and does the following only when it finds the legacy layout:

1. Generates the dashboard API token in `/etc/kb/api.env` if absent (Section 8.5) and switches an existing `operator` from a `nologin` shell to `/bin/sh`.
2. Moves `/etc/ssh/sshd_config.d/kb-operator.conf` to `/etc/kb/sshd_kb-operator.conf.legacy`. It is kept as a backup, never deleted, and your edits are not lost; the *packaged* `sshd_kb-operator.conf` stays in force because its `ForceCommand` points at the packaged wrapper.
3. Moves legacy dev units (`kbd`, `kb-sensor`, `kb-checker`, `kbopd`, `kbagents`, `sshd@kb-operator`) out of `/etc/systemd/system` when they reference `/usr/local/bin` or the old sshd config path. They are kept as `.pre-migration` and the packaged units apply instead.
4. Runs `systemctl daemon-reload` and *reloads* the host's own `sshd` (`ssh` on Ubuntu, `sshd` on Arch), which releases the port 2222 it had wrongly bound. Reload, not restart: your admin session stays up.
5. Restarts `sshd@kb-operator` unless you stopped it by hand (it is neither enabled nor active); in that case it is left exactly as you left it.
6. Prints a notice if `kbd` or another migrated unit is running with the old code or without the new token. It does not restart them for you; run `sudo kbctl down && sudo kbctl up`.

Separately, on every install and upgrade it removes any boot autostart of `kb-checker` (and stops it on a fresh install), so the watchdog stays dead unless you start it yourself (Chapter 12). Units from a legacy install that live in `/etc/systemd/system` are retired by the same step, so the packaged units take effect.

WHY THE OPERATOR ACCOUNT HAS A REAL SHELL

OpenSSH runs a `ForceCommand` as `$SHELL -c ...`, and a `nologin` shell refuses it ("This account is currently not available"). `provision.sh` therefore creates `operator` with `/bin/sh`, and the migration step switches an existing `operator` that has a `nologin` or `false` shell. It also sets the account's shadow password to `*`: `useradd` leaves it as `!` (locked), and OpenSSH without PAM refuses a locked account even for key logins. The account stays locked down by the operator `sshd` config: key-only, the wrapper forced, no forwarding, no password. Chapter 9 has the one-command check.

6.3 Verify the installation

1 The files are where they should be

```
$ pacman -Ql kernel-borderlands | grep -E '/usr/bin/|/etc/kb/'
```

2 **The units are known to systemd and use the right paths**

```
$ systemctl cat kbd.service | grep -E 'ExecStart|Environment'
ExecStart=/usr/bin/kbd --db /var/lib/kbd/state.db --policy /etc/kb/policy.yaml \
          --rules /etc/kb/rules.yaml --workloads /etc/kb/workloads.yaml
Environment=KB_HTTP_CORS_ORIGIN=http://127.0.0.1:8082,http://localhost:8082
$ systemd-analyze verify kbd.service kb-sensor.service kbopd.service
```

3 **The CLI answers and reports the maintenance hold**

```
$ kbctl status
kbd.service                  inactive
kb-sensor.service            inactive
kbopd.service                inactive
sshd@kb-operator.service     inactive
kb-checker.service           UNDER MAINTENANCE AND REPAIRS (kept down)
```

4 **The provisioning result**

```
$ getent passwd operator
operator:x:998:994::/home/operator:/bin/sh
$ sudo ls -l /etc/kb /etc/kb/ssh
```

6.4 Upgrade

Install the new file the same way (`pacman -U`). Because the five config files are in the `backup=()` array, a locally modified file is *kept* and the new default is written beside it as `.pacnew`. Merge with `pacdiff` (from `pacman-contrib`). Restart what changed: `sudo systemctl daemon-reload && sudo kbctl down && sudo kbctl up`. The SQLite state in `/var/lib/kbd/` is untouched by an upgrade.

6.5 Remove

```
$ sudo pacman -R kernel-borderlands
```

Before removal the units are disabled and stopped. These are intentionally left behind, even after a purge, because they hold credentials and audit state an operator would not expect a package manager to destroy: `/etc/kb/ssh/` and `/etc/kb/authorized_keys`, `/var/lib/kbd/state.db`, the `kb` group and the `operator` user. To remove them as well:

```
$ sudo rm -rf /etc/kb /var/lib/kbd
$ sudo userdel operator && sudo groupdel kb
```

IF A *DEVELOPMENT* INSTALL ALREADY EXISTS ON THE HOST

`install.sh` and `sudo make install` place binaries in `/usr/local/bin` and units in `/etc/systemd/system`. Units in `/etc/systemd/system` override the package's units in `/usr/lib/systemd/system`, and `/usr/local/bin` wins over `/usr/bin` on `PATH`. Installing the package used to keep running the old code silently. Now the migration step moves the legacy *units* aside (Section 6.2), so the packaged units apply. The old binaries in `/usr/local/bin` remain and still win on an interactive `PATH`, though the packaged units call `/usr/bin/...` by absolute path. Delete the old `/usr/local/bin/kb*` files to avoid running a stale `kbctl` by hand. Restart the stack once (`sudo kbctl down && sudo kbctl up`) so the running units adopt the packaged ones.

PART III

OPERATIONS

CHAPTER 7

STARTING THE STACK

7.1 The boot order

Four units make up the running system. Their order is not arbitrary: `kbd` must bind its sockets *before* anything tries to connect to them.

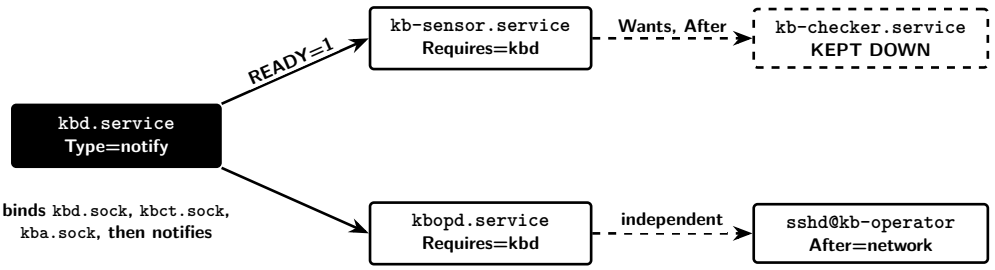


Figure 7.1: Start order. `kbd` sends `sd_notify(READY=1)` only after its sockets exist, so the units that `Require` it never race ahead of it. The dashed box is the unit held down by the maintenance condition.

Unit	Runs	Notes
kbd.service	kbd with --db /var/lib/kbd/state.db and --policy, --rules, --workloads from /etc/kb	Type=notify; RuntimeDirectory=kb; StateDirectory=kbd; sets KB_HTTP_CORS_ORIGIN; loads /etc/kb/api.env; after readiness runs chgrp kb /run/kb/kba.sock. Conservative hardening only.
kb-sensor.service	kbd_sensor	Root, no capability dropping (needs CAP_BPF/PERFMON/SYS_ADMIN). Loads the eBPF hooks; connects to kbd.sock.
kbopd.service	kbopd headless	Serves the dashboard on 127.0.0.1:8082 and spawns mcp-server. Fully hardened (ProtectSystem=strict, empty capability set).
sshd@kb-operator.service	sshd -D -f /etc/kb/sshd_kb-operator.conf	Dedicated instance on port 2222, key-only, forces the wrapper script. Runs with a minimal capability set (CAP_SYS_CHROOT, SETUID, SETGID, CHOWN, KILL, AUDIT_WRITE); an empty set breaks sshd's privilege separation.
kb-checker.service	kb-checker monitor --all	Held down: never enabled or auto-started; runs only if you start it by hand.
kbagents.service	kbagents	Installed, not enabled; see Chapter 15.

7.2 The one-command start

1 Authorize your SSH public key first (once)

```
$ sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub
key authorized for operator
```

2 Bring the stack up

```
$ sudo kbctl up
```

This runs `systemctl enable --now kbd.service kb-sensor.service kbopd.service sshd@kb-operator.service` in that order. `kbctl` adds `sudo` itself when you are not root, so plain `kbctl up` also works and prompts for your password.

3 Confirm

```
$ kbctl status
kbd.service                 active
kb-sensor.service           active
kbopd.service               active
sshd@kb-operator.service    active
kb-checker.service          UNDER MAINTENANCE AND REPAIRS (kept down)
```

4 Open a surface

```
$ kbctl dashboard open        # web, http://127.0.0.1:8082
$ kbctl ssh connect           # TUI over the operator sshd
$ kbctl tui                   # TUI directly on this terminal
```

7.3 The same thing with `systemctl`

Every `kbctl` verb is a thin wrapper. Know the equivalents; they are what you use when `kbd` is down and `kbctl` cannot dial it.

kbctl	systemctl equivalent
`kbctl up`	`systemctl enable --now kbd kb-sensor kbopd sshd@kb-operator`
`kbctl down`	`systemctl stop` on those four, in reverse order (does not disable)
`kbctl status`	`systemctl is-active` on each unit, plus the maintenance label
`kbctl dashboard up / down`	`systemctl enable --now / disable --now kbopd`
`kbctl ssh up / down`	`systemctl enable --now / disable --now sshd@kb-operator`

NOTE

`up` ***enables*** **units (they start at every boot).** `down` **only** ***stops*** **the whole stack and leaves it enabled, so a reboot brings it back. To keep a unit down across reboots use** `kbctl dashboard down` **or** `kbctl ssh down`**, which also** ***disable*** **it.**

7.4 Watching a start succeed

```
$ journalctl -u kbd -u kb-sensor -u kbopd -f
... kbd: [KB] gRPC health service registered, status=SERVING for "kernel-borderlands"
... kbd: [KB] audit log hash chain verified intact (0 entries)
... kbd: [KB] Control plane ready
... kb-sensor: (6 hooks attached; uprobes into libssl/libgnutls)
... kbopd: Vite Dashboard Server listening on http://localhost:8082
```

`kbd` **printing** `audit log hash chain verified intact` **on every start is the cheapest tamper check you will ever get; see Section 11.6.**

7.5 Stopping, restarting, and what to expect

You do	What happens
`sudo systemctl restart kbd`	`kb-sensor` and `kbopd` (which `Require` it) are restarted with it; the sensor reconnects and `kbd` re-pushes the rules and sensitive paths at connect time.
`sudo systemctl stop kb-sensor`	Telemetry stops; existing containment stays in the kernel maps until cleared. `kbd` keeps serving history.
`sudo kbctl down`	Orderly stop in reverse boot order.
Power loss	`kbd`'s SQLite WAL replays on next start (`L1 restored from L2`).

CHAPTER 8

THE WEB DASHBOARD

8.1 How it is put together

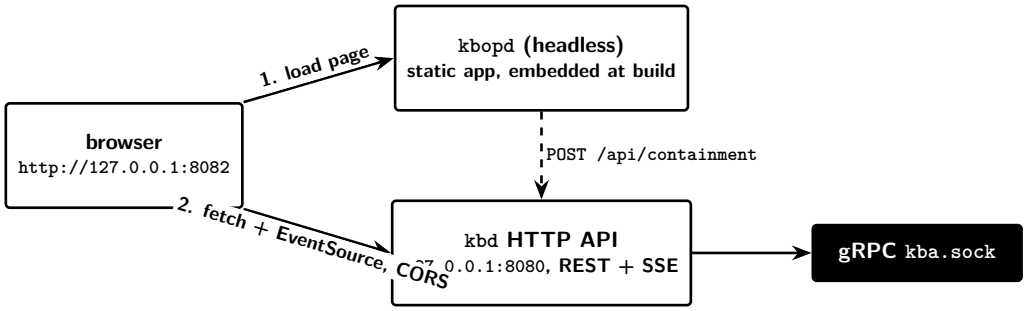


Figure 8.1: The page itself comes from `kbopd` on 8082; every data call goes from the *browser* straight to `kbd` on 8080. That second hop is cross-origin and is why `KB_HTTP_CORS_ORIGIN` exists.

The dashboard's JavaScript builds its API address as `http://<the host you loaded it from>:8080`. Loaded via `127.0.0.1` it calls `127.0.0.1:8080`; loaded via `localhost` it calls `localhost:8080`. Both origins are in the packaged allowlist; any other hostname is blocked by the browser until you add it.

8.2 Commands

```
$ kbctl dashboard up        # enable + start kbopd, wait for the port, print the URL
dashboard up: http://127.0.0.1:8082
$ kbctl dashboard open      # up (if needed) + launch the default browser
opened http://127.0.0.1:8082
$ kbctl dashboard status    # exit 0 if active AND listening, 3 otherwise
kbopd.service: active, http://127.0.0.1:8082 listening=true
$ kbctl dashboard url       # just print it, handy in scripts
$ kbctl dashboard down      # stop + disable
```

`up` polls the port for up to 15 seconds. If no browser can be launched (a headless server), `open`

prints the URL and succeeds; open it from another machine through the tunnel below.

8.3 Reaching it from your workstation

The dashboard binds loopback only. The operator `sshd` deliberately forbids TCP forwarding (`AllowTcpForwarding no`), so tunnel through your *normal* SSH service (port 22):

```
workstation$ ssh -N -L 8082:127.0.0.1:8082 -L 8080:127.0.0.1:8080 you@kb-host
workstation$ xdg-open http://127.0.0.1:8082
```

Forward *both* ports: the page comes from 8082, the data from 8080. Use `127.0.0.1` (or `localhost`) in the browser, matching the CORS allowlist.

8.4 What the API exposes

Route on :8080	Auth	Purpose
/api/processes	none	Tracked processes and zones.
/api/alerts, /api/logs	none	Alert stream snapshot, log tail.
/api/services	none	Health of KB services.
/api/metrics	none	Event rates, active process counts.
/api/agents	none	Proxied from the swarm status server (:8601).
/api/policy	none	Current policy.
/api/audit/verify	none	Hash-chain verification.
/api/events	none	Server-Sent Events live stream.
/api/isolate, /api/restore	token	Contain / release a process.
/api/policy/reload	token	Reload `policy.yaml`.

8.5 The API token: how the action buttons are authorized

The three mutating routes are wrapped in `requireAPIToken` and fail closed: without a configured token they are unusable, and with one they need a matching `Authorization: Bearer` header. The package sets this up so the buttons work without weakening that check:

1. **Generation.** On install, `/usr/lib/kb/migrate-legacy.sh` creates `/etc/kb/api.env` containing `KB_HTTP_API_TOKEN=`*64 random hex characters* (mode 0640, `root:kb`). It is created once and never overwritten.
2. **Server.** `kbd.service` loads it with `EnvironmentFile=-/etc/kb/api.env`. `kbd` must be *restarted* to read a newly created token; the installer prints a notice and does not restart `kbd` for you.

3. **Browser.** `kbctl dashboard open` **opens** `http://127.0.0.1:8082/#token=...`. **A URL *fragment* is never sent to any server. The page moves it into** `sessionStorage` **(this tab only, cleared when the tab closes) and removes it from the address bar. It is *not* compiled into the dashboard bundle, so loading the page without the fragment gives read-only views.**

```
$ kbctl dashboard open              # token handed to the browser automatically
$ kbctl dashboard url --token       # print the tokened URL for another browser; treat as a secret
$ sudo cat /etc/kb/api.env          # the token itself (root or the kb group)
```

LIMITS OF THIS DESIGN

Anyone who can read `/etc/kb/api.env` **(root and the** `kb` **group, which includes** `operator`**) can isolate processes through the HTTP API; so can anyone who sees the tokened URL (it appears in the process list of the machine while the browser launches). The API listens on loopback only. To rotate the token, delete** `/etc/kb/api.env`**, run** `sudo /usr/lib/kb/migrate-legacy.sh upgrade`**, then** `sudo systemctl restart kbd` **and reopen the dashboard. If the buttons report "API token missing or wrong", you opened the page without the fragment (use** `kbctl dashboard open`**) or** `kbd` **has not been restarted since the token was created.**

VERIFICATION STATUS

The dashboard compiles with the new logic, the served bundle contains it, the fragment parsing was tested, and `kbctl dashboard url --token` **was tested against a token file. It was not exercised in a real browser against a live** `kbd` **(no browser extension and no root in the build sessions); step D of the acceptance checklist closes that.**

8.6 Symptoms and causes

You see	Cause and fix
Page loads, every panel empty, console shows CORS errors	The page's origin is not in `KB_HTTP_CORS_ORIGIN`. Load via `127.0.0.1` or `localhost`, or add your origin in a drop-in and restart `kbd`.
`connection refused` on 8082	`kbopd.service` is not running: `kbctl dashboard status`.
Isolate / restore / reload report "API token missing or wrong"	Page opened without the token fragment, or `kbd` not restarted since `/etc/kb/api.env` was created: `kbctl dashboard open`; `sudo systemctl restart kbd`.
Page loads but data source shows a different host	You loaded it via a hostname not in the allowlist; the dashboard derives the API host from the page URL.
`kbctl dashboard up` times out	`journalctl -u kbopd`. Another process already holds 8082.

CHAPTER 9

THE SSH TERMINAL CONSOLE

9.1 The architecture

The console is not a feature of `kbd`. It is a *second, independent* `sshd` that exists only to authenticate an operator and force one program. Host keys, `authorized_keys` and PTY handling all live in that `sshd`.

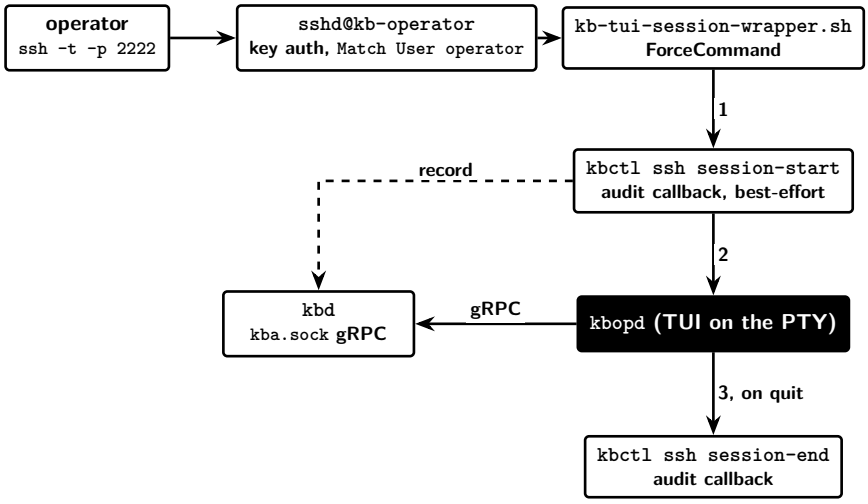


Figure 9.1: One SSH session. The wrapper records the start, runs the TUI, records the end, and returns the TUI's exit status to the client.

Both audit callbacks are best-effort by design. If `kbd` is down, `kbctl ssh session-start` prints a warning to the operator's terminal and exits 0; an audit-logging outage must never become a denial of access to the very tool you need during an incident.

9.2 Commands

```
$ sudo kbctl ssh up            # enable + start sshd@kb-operator, wait for :2222
$ kbctl ssh status             # exit 0 only if active AND port answering
$ sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub
```

```
key authorized for operator
$ kbctl ssh connect            # to 127.0.0.1:2222 as operator, starting it if needed
$ kbctl ssh connect kb-host    # to a remote host
$ kbctl ssh connect kb-host -i ~/.ssh/ops_key --port 2222
$ sudo kbctl ssh down          # stop + disable
```

`ssh connect` replaces itself with `ssh -t` (so your terminal, window resizes and exit status are real SSH's). For loopback targets it adds `UserKnownHostsFile=/dev/null` and `StrictHostKeyChecking=no`, because the host key is regenerated per machine and would otherwise pollute `known_hosts`; *remote* hosts keep normal host-key checking.

9.2.1 `add-key` in detail

It accepts a public-key file or `-` for stdin, requires exactly one line, checks the key type prefix (`ssh-ed25519`, `ssh-rsa`, `ecdsa-sha2-*`, `sk-*`), skips a key already present, and appends to `/etc/kb/authorized_keys` with `tee -a` under `sudo` (the file is `operator:kb` 0600).

```
$ ssh-add -L | head -1 | sudo kbctl ssh add-key -     # from an agent
$ printf 'garbage\n' | kbctl ssh add-key -
error: not an OpenSSH public key (want ssh-ed25519/ecdsa-sha2-*/sk-*/ssh-rsa ...)
```

9.3 The sshd configuration, line by line

Listing 9.1: `/etc/kb/sshd_kb-operator.conf` (packaged)

```
# /etc/kb/sshd_kb-operator.conf  (NOT /etc/ssh/sshd_config.d/ - the host's sshd includes that
# directory and would bind Port 2222 itself, starving this instance)
# Loaded by the dedicated `sshd@kb-operator` instance below, NOT the host's
# normal admin sshd on port 22 - keeps operator access to kb-tui fully
# isolated from general host admin access.
Port 2222
# ProtectSystem=strict makes /run read-only, so sshd could not write /run/sshd.pid; systemd
# tracks the PID itself. PrintLastLog would try to write /var/log/lastlog (read-only too).
PidFile none
PrintLastLog no
HostKey /etc/kb/ssh/ssh_host_ed25519_key

Match User operator
    ForceCommand /usr/local/bin/kb-tui-session-wrapper.sh
    AuthorizedKeysFile /etc/kb/authorized_keys
    X11Forwarding no
    AllowTcpForwarding no
    PermitTTY yes
```

Directive	Effect
`Port 2222`	Dedicated port, separate from the host's port 22.
`HostKey /etc/kb/ssh/...`	Host key private to this instance.
`Match User operator`	Everything below applies only to the `operator` account.
`ForceCommand ...wrapper.sh`	Whatever the client asks to run, this runs instead.
`AuthorizedKeysFile /etc/kb/authorized_keys`	Keys live outside any home directory.
`X11Forwarding no, AllowTcpForwarding no`	No tunnels through the console.
`PermitTTY yes`	The TUI needs a PTY.
`PidFile none`	The unit makes `/run` read-only (`ProtectSystem=strict`), so sshd cannot write a PID file; systemd tracks the process itself.
`PrintLastLog no`	`/var/log/lastlog` is read-only for the same reason.

HARDENING YOU MAY WANT TO ADD

The packaged file does not disable password authentication for non-`operator` users. The `operator` account has no password, so it is moot for that account, but adding `PasswordAuthentication no` and `KbdInteractiveAuthentication no` makes the whole instance key-only. Edit `/etc/kb/sshd_kb-operator.conf`; it is a protected config file and survives upgrades.

9.4 Check the console on your host

OpenSSH runs a `ForceCommand` through the account's login shell, which a `nologin` shell refuses. `provision.sh` therefore creates `operator` with `/bin/sh`, and the package's migration step fixes an existing account that still has `nologin` or `false`. This was verified on the Ubuntu 24.04 VM (key accepted, TUI rendered, live link to `kbd`); run the same check once on your own host after `kbctl up`:

1 **Connect**

```
$ kbctl ssh connect
```

2 **Read the result**

- The TUI appears. Nothing to do.
- "This account is currently not available." and the connection closes: the account still has a `nologin` shell (for example the package was installed before this fix and the migration did not run). Fix it as below.

3 **If refused, give the account a plain shell by hand**

```
$ sudo usermod --shell /bin/sh operator
$ kbctl ssh connect
```

This is safe in this design: the only way in is a key in `/etc/kb/authorized_keys`, the config forces the wrapper and forbids forwarding, and the account has no password.

9.5 Using the TUI

9.5.1 The four tabs

`Tab` moves forward, `Shift+Tab` backward. `?` toggles help (`Esc` also closes it). `q` or `Ctrl+C` quits.

Tab	Keys	Meaning
Processes	`j`/`k` or arrows	Move selection through the process table (PID, COMM, ZONE, SCORE, UID, CONTAINMENT).
	`/`	Start a filter; type to narrow, `Enter`/`Esc` to stop typing, `Esc` again clears it.
	`c`	Contain the selected process (see below).
Alerts	`j`/`k`, `Enter`	Scroll alerts; `Enter` expands or collapses one.
Agent Activity	`j`/`k`	Scroll the swarm's activity feed.
Query Console	type, `Enter`	Run a command; `Up`/`Down` history; `Ctrl+U` clears the line.

9.5.2 Containing a process

1. Select it and press `c`.

A modal asks for a level.

2. Pick the level.

Key	Level
0	None (release)
1	Cgroup
2	Seccomp
3	Namespace
4	Terminate

3. Confirm.

`y` or `Enter` submits with the reason `operator action via kb-tui`; `n` returns to level selection; `Esc` cancels.

9.5.3 The query console

```
> help
> pid 4242                    # full state for one process
```

```
> zone borderlands              # list processes in safe|suspicious|borderlands
> stats                         # global telemetry snapshot
> contain 4242 seccomp suspicious beaconing
> clear
```

Levels for `contain`: `none`, `cgroup`, `seccomp`, `namespace`, `terminate`.

9.5.4 When `kbd` is unreachable

If `/run/kb/kba.sock` cannot be dialed the TUI does *not* fail. It shows an `[OFFLINE]` banner (“`kbd unreachable`”) and a `[DEMO DATA]` table so you can see the interface working. Never act on demo rows: the PIDs are fictional. Read the banner before the table.

CHAPTER 10

KBCTL REFERENCE

`kbctl` is the programmatic workhorse. Every control operation is a typed protobuf gRPC request over `/run/kb/kba.sock`. The reference text below is generated from the installed binary's `--help` by `latex/field-manual/gen-help.sh`, so it cannot drift from the code.

10.1 Command map

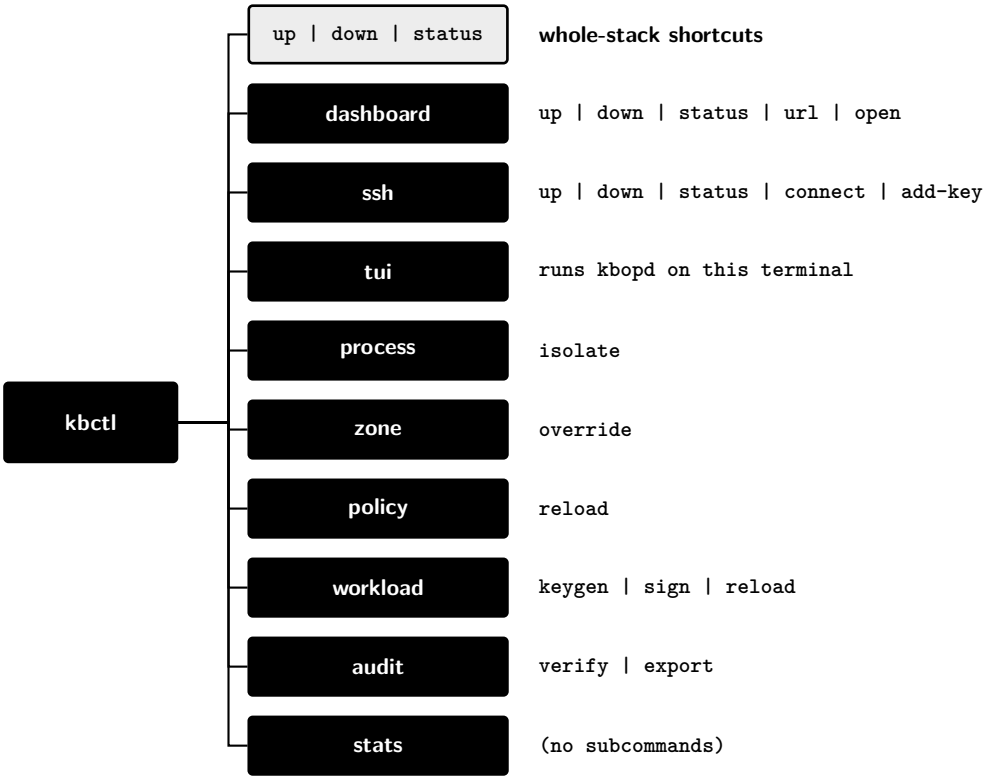


Figure 10.1: Everything under `kbctl`. The first row are top-level shortcuts that act on the whole stack.

10.2 Global flag

```
--socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

Only commands that talk to `kbd` use it (`policy`, `workload reload`, `zone`, `process`, `audit`, `stats`). The service-control commands (`up`, `down`, `status`, `dashboard`, `ssh`) go through `systemctl` and work even when `kbd` is down.

10.3 Exit codes

Code	Meaning
0	Success. For `dashboard status` and `ssh status`: the unit is active *and* the port answers.
1	Error (printed as `error: ...`): bad arguments, RPC failure, `systemctl` failure.
3	`dashboard status` / `ssh status` only: unit not active or port not listening. Use it in scripts.

10.4 Stack control

10.4.1 kbctl up

```
Enable and start the stack (kbd, sensor, dashboard, SSH console; kb-checker stays down for
    • maintenance)

Usage:
  kbctl up [flags]

Flags:
  -h, --help   help for up

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.4.2 kbctl down

```
Stop the whole stack in reverse boot order (does not disable it)

Usage:
  kbctl down [flags]

Flags:
```

```
  -h, --help   help for down

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.4.3 kbctl status

```
Show the state of every Kernel Borderlands unit

Usage:
  kbctl status [flags]

Flags:
  -h, --help   help for status

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.5 Web dashboard

10.5.1 kbctl dashboard

```
Manage the web dashboard served by kbopd, the unified kb-tui binary that
embeds kb-dashboard's production build. It listens on 127.0.0.1 only; reach it
from another machine with an SSH local forward or use the SSH TUI
('kbctl ssh connect') instead.

Usage:
  kbctl dashboard [command]

Available Commands:
  down        Stop and disable the web dashboard
  open        Start the dashboard if needed and open it in the default browser
  status      Show whether the dashboard unit is running and its port is answering
  up          Enable and start the web dashboard, then print its URL
  url         Print the dashboard URL (--token adds the API token fragment; treat the output as a
    • secret)

Flags:
  -h, --help       help for dashboard
      --port int   dashboard port (kbopd's embedded server is fixed at 8082; change only if you
    • rebuilt it) (default 8082)

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")

Use "kbctl dashboard [command] --help" for more information about a command.
```

10.5.2 kbctl dashboard open

```
Start the dashboard if needed and open it in the default browser

Usage:
  kbctl dashboard open [flags]

Flags:
  -h, --help   help for open

Global Flags:
      --port int        dashboard port (kbopd's embedded server is fixed at 8082; change only if you
    • rebuilt it) (default 8082)
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.5.3 kbctl dashboard status

```
Show whether the dashboard unit is running and its port is answering

Usage:
  kbctl dashboard status [flags]

Flags:
  -h, --help   help for status

Global Flags:
      --port int        dashboard port (kbopd's embedded server is fixed at 8082; change only if you
    • rebuilt it) (default 8082)
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.5.4 kbctl dashboard url

```
Print the dashboard URL (--token adds the API token fragment; treat the output as a secret)

Usage:
  kbctl dashboard url [flags]

Flags:
  -h, --help    help for url
      --token   include the API token as a URL fragment

Global Flags:
      --port int        dashboard port (kbopd's embedded server is fixed at 8082; change only if you
    • rebuilt it) (default 8082)
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.6 SSH console and TUI

10.6.1 kbctl ssh

```
Manage and connect to the SSH operator console (sshd@kb-operator, port 2222)

Usage:
  kbctl ssh [command]

Available Commands:
  add-key       Authorize an operator public key for the SSH console (/etc/kb/authorized_keys)
  connect       Open the TUI over SSH on the operator console (default host 127.0.0.1)
  down          Stop and disable the operator sshd
  status        Show whether the operator sshd is running and its port is answering
  up            Enable and start the operator sshd (sshd@kb-operator.service)

Flags:
  -h, --help          help for ssh
      --port int      operator sshd port (packaging/ssh/kb-operator.conf) (default 2222)
      --user string   account the console ForceCommand is bound to (default "operator")

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")

Use "kbctl ssh [command] --help" for more information about a command.
```

10.6.2 kbctl ssh connect

```
Replaces this process with ssh to the operator sshd, which ForceCommands the
audit wrapper and then kbopd. Starts the console first if it is not running
and the target is this machine.

Usage:
  kbctl ssh connect [host] [flags]

Flags:
  -h, --help              help for connect
  -i, --identity string   private key file passed to ssh -i

Global Flags:
      --port int        operator sshd port (packaging/ssh/kb-operator.conf) (default 2222)
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
      --user string     account the console ForceCommand is bound to (default "operator")
```

10.6.3 kbctl ssh add-key

```
Authorize an operator public key for the SSH console (/etc/kb/authorized_keys)
```

```
Usage:
  kbctl ssh add-key <pubkey-file|-> [flags]

Flags:
  -h, --help   help for add-key

Global Flags:
      --port int        operator sshd port (packaging/ssh/kb-operator.conf) (default 2222)
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
      --user string     account the console ForceCommand is bound to (default "operator")
```

10.6.4 kbctl tui

```
Replaces this process with kbopd attached to your terminal, which renders the
ratatui console. For a remote session through the dedicated operator sshd
(port 2222) use 'kbctl ssh connect'.

Usage:
  kbctl tui [flags]

Flags:
  -h, --help   help for tui

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.7 Containment and classification

10.7.1 kbctl process isolate

```
Sends a containment command to the kernel sensor via kbd's
SetContainment RPC. Defaults to NAMESPACE-level isolation; pass --level
to choose CGROUP, SECCOMP, NAMESPACE, or TERMINATE explicitly.

Usage:
  kbctl process isolate [flags]

Flags:
  -h, --help            help for isolate
      --level string    CGROUP | SECCOMP | NAMESPACE | TERMINATE (default "NAMESPACE")
      --pid uint32      target process PID (required)
      --reason string   operator-supplied reason, recorded in the audit log (default "kbctl process
    • isolate")

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

Levels are `CGROUP`, `SECCOMP`, `NAMESPACE` (default) and `TERMINATE`, in increasing severity. The

command prints `pid=N contained at level L`. The reason is written to the audit log.

10.7.2 kbctl zone override

```
Relabels how kbd tracks a process's zone (SAFE / SUSPICIOUS /
BORDERLANDS) for display and classification purposes only - it does not
touch kernel or enforcement state. A subsequent real zone transition
event from the sensor will overwrite this the next time the process's
score crosses a threshold. The override is recorded in the audit log.

Usage:
  kbctl zone override [flags]

Flags:
  -h, --help            help for override
      --pid uint32      target process PID (required)
      --reason string   operator-supplied reason, recorded in the audit log
      --zone string     SAFE | SUSPICIOUS | BORDERLANDS (required)

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

OVERRIDE IS COSMETIC, NOT ENFORCEMENT

`zone override` relabels how `kbd` *displays and classifies* a process. It does not touch kernel or enforcement state, and the next real zone transition from the sensor overwrites it. To actually restrict a process, use `process isolate`.

10.8 Policy, workloads, audit, stats

10.8.1 kbctl policy reload

```
Reload policy.yaml from the path kbd was started with

Usage:
  kbctl policy reload [flags]

Flags:
  -h, --help   help for reload

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.2 kbctl workload keygen

```
Generate an Ed25519 keypair for signing/verifying workloads.yaml
```

```
Usage:
  kbctl workload keygen [flags]

Flags:
  -h, --help             help for keygen
      --name string      base filename for the generated <name>.key/<name>.pub (default "workloads-
    • signer")
      --out-dir string   directory to write the generated key pair into (default ".")

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.3 kbctl workload sign

```
Sign a workloads.yaml file for kbd's --workloads-pubkey verification (CWP.md S.4.3/S.11.4)

Usage:
  kbctl workload sign <workloads.yaml> [flags]

Flags:
  -h, --help               help for sign
      --key string         path to the Ed25519 private key to sign with (required)
      --signed-by string   identity recorded in policy_metadata.signed_by (CWP.md S.12.1)
      --version int        policy_metadata.version to record (default 1)

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.4 kbctl workload reload

```
Reload workloads.yaml and push the registry live to every connected sensor

Usage:
  kbctl workload reload [flags]

Flags:
  -h, --help   help for reload

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.5 kbctl audit verify

```
Verify the audit log hash chain has not been tampered with

Usage:
  kbctl audit verify [flags]

Flags:
```

```
  -h, --help   help for verify

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.6 kbctl audit export

```
Export the full hash-chained audit log as JSON

Usage:
  kbctl audit export [flags]

Flags:
  -h, --help         help for export
      --out string   output file (JSON); defaults to stdout

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.8.7 kbctl stats

```
Fetch global telemetry volumes and active process counts

Usage:
  kbctl stats [flags]

Flags:
  -h, --help   help for stats

Global Flags:
      --socket string   path to kbd's gRPC Unix domain socket (default "/run/kb/kba.sock")
```

10.9 The daemons' own flags

10.9.1 kbd

```
kbd is the control plane daemon for Kernel Borderlands.
It aggregates eBPF events, manages process behavioral state,
and coordinates with the AADS agent swarm.

Usage:
  kbd [flags]

Flags:
  -d, --db string                                path to SQLite state database (L2 durable store) (
    • default "/var/lib/kbd/state.db")
```

```
      --grpc-socket string                    path to the gRPC UDS socket (kba.sock) (default "/run
    • /kb/kba.sock")
  -h, --help                                  help for kbd
      --http-addr string                      address for the HTTP API/SSE server (web dashboard)
    • to bind - loopback-only by default (default "127.0.0.1:8080")
  -p, --policy string                         path to policy.yaml (per-process thresholds, auto-
    • terminate rules) (default "config/policy.yaml")
      --rules string                          path to rules.yaml (dynamic attack-chain rules pushed
    •  to the sensor at connect time); empty disables the push, sensor falls back to compiled-in
    • default rules (default "config/rules.yaml")
      --workloads string                      path to workloads.yaml (CWP protected-workload
    • registry, docs/features/CWP.md); empty disables CWP entirely. A missing file is not an error
    • - CWP just has nothing registered (default "config/workloads.yaml")
      --workloads-pubkey kbctl workload sign   path to an Ed25519 public key (hex/base64/raw) that --
    • workloads must be validly signed against (CWP.md S.4.3/S.11.4, see kbctl workload sign); empty
    •  (default) keeps CWP unsigned/opt-in
```

10.9.2 kb-checker (kept down; for reference)

```
Start the background validation loops daemon

Usage: kb-checker monitor [OPTIONS]

Options:
      --all                        Monitor all loops simultaneously
      --grpc-socket <GRPC_SOCKET>  Optional: Path to run the diagnostic gRPC server on Unix Domain
    • Socket [default: /run/kb/kbc.sock]
  -h, --help                       Print help
```

CHAPTER 11

COMMAND COMBINATIONS AND PLAYBOOKS

A command is a word; a playbook is a sentence. Each recipe below is complete, ordered, and says what you should see after every step.

11.1 Playbook 1: cold start to first dashboard

```
$ sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub     # once per operator
$ sudo kbctl up
$ kbctl status                                      # all four active
$ kbctl dashboard open                              # browser appears
$ kbctl stats
events/sec:       112.40
active processes: 238
```

If `stats` **errors with** `dial unix /run/kb/kba.sock: no such file`, `kbd` **is not running:** `systemctl status kbd`.

11.2 Playbook 2: a remote operator takes over

On the server (once): `sudo kbctl ssh add-key` **for their key,** `sudo kbctl ssh up`. **On their laptop:**

```
$ ssh -t -p 2222 operator@kb-host        # lands directly in the TUI, nothing else
```

They need no account on the server and get no shell. Revoke by deleting their line from `/etc/kb/authorized_keys`**; no restart is needed.**

11.3 Playbook 3: incident, from detection to containment

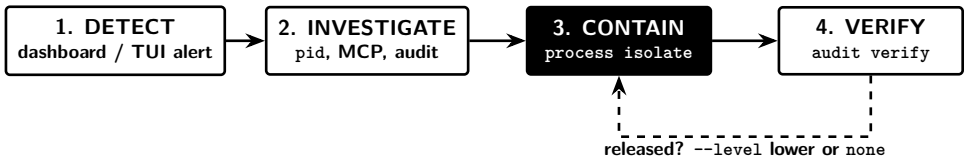


Figure 11.1: The response loop. Containment is reversible until you choose `TERMINATE`.

1 Detect.

A process enters `BORDERLANDS`. The dashboard raises a `BORDERLANDS_ENTRY` alert.

2 Investigate.

In the TUI Query Console: `pid 4242`, `zone borderlands`.

3 Contain, mildest effective level first.

```
$ sudo kbctl process isolate --pid 4242 --level SECCOMP \
      --reason "beaconing to unknown ASN, ticket 1187"
pid=4242 contained at level SECCOMP
```

Escalate to `NAMESPACE` only if the behaviour continues; reserve `TERMINATE` for confirmed hostile code.

4 Verify the record.

```
$ kbctl audit verify
chain intact - 1432 entries verified     # counts will differ
```

11.4 Playbook 4: change policy without restarting

```
$ sudo $EDITOR /etc/kb/policy.yaml
$ sudo kbctl policy reload
```

`kbd` re-reads the file it was *started with* (`/etc/kb/policy.yaml` in the packaged unit) and logs how many process policies and extra sensitive paths it loaded.

11.5 Playbook 5: protect a workload with signed CWP rules

Protected-workload rules can require a signature so nobody can weaken them by editing a file.

1 Create a signing key pair somewhere safe (not on the monitored host if you can avoid it).

```
$ kbctl workload keygen --out-dir ~/kb-keys --name cwp-signer
Wrote private key (keep secret): /home/you/kb-keys/cwp-signer.key
Wrote public key (for kbd's --workloads-pubkey): /home/you/kb-keys/cwp-signer.pub
```

2 Edit and sign. The file is rewritten *in place*.

```
$ sudo $EDITOR /etc/kb/workloads.yaml
$ sudo kbctl workload sign /etc/kb/workloads.yaml \
      --key ~/kb-keys/cwp-signer.key --signed-by "alice@soc" --version 2
Signed /etc/kb/workloads.yaml (signed_by="alice@soc" version=2)
```

3 Tell `kbd` to require the signature.

Install the public key and add a drop-in (Section 14.7):

```
$ sudo install -m644 ~/kb-keys/cwp-signer.pub /etc/kb/workloads.pub
$ sudo systemctl edit kbd
[Service]
ExecStart=
ExecStart=/usr/bin/kbd --db /var/lib/kbd/state.db --policy /etc/kb/policy.yaml \
    --rules /etc/kb/rules.yaml --workloads /etc/kb/workloads.yaml \
    --workloads-pubkey /etc/kb/workloads.pub
$ sudo systemctl restart kbd
```

4 Push live to every connected sensor.

```
$ sudo kbctl workload reload
```

Without `--workloads-pubkey`, CWP stays unsigned and opt-in; an unsigned file is accepted.

11.6 Playbook 6: prove the audit log is intact

```
$ kbctl audit verify
chain intact - 1432 entries verified          # counts will differ; exit 0
$ kbctl audit export --out /tmp/audit-$(date +%F).json
wrote 1432 entries to /tmp/audit-2026-10-27.json
```

Each entry hashes the previous one (SHA-256). A broken chain prints `CHAIN BROKEN - N entries verified before break`. Export before you investigate a suspected break so you hold a copy of the evidence.

11.7 Playbook 7: scripting and health gates

```
# wait until the SSH console really answers, then continue
$ until kbctl ssh status >/dev/null; do sleep 1; done

# fail a CI step if the dashboard is not serving
$ kbctl dashboard status || { echo "dashboard down"; exit 1; }

# watch the stack
$ watch -n2 kbctl status

# stop everything for maintenance, start it again
$ sudo kbctl down && sudo kbctl up

# one-liner health report
$ kbctl status; kbctl stats; kbctl audit verify
```

Exit code 3 from the two `status` commands means "not up"; code 1 means the command itself failed.

11.8 Playbook 8: a quick demo for an audience

The short version. The full, timed, install-to-uninstall demonstration with expected output and status tags is Chapter 13.

```
$ sudo kbctl up
$ kbctl dashboard open                      # screen 1: live dashboard
$ kbctl ssh connect                         # screen 2: the same data in a terminal
# in the TUI: Tab to Processes, press / to filter, c to contain a demo process
$ kbctl audit verify                        # screen 3: the audit proof
```

Remember the TUI's `[DEMO DATA]` banner when `kbd` is down: do not present fictional rows as live.

11.9 Combination cheat sheet

Goal	Command
Everything up and verified	`sudo kbctl up && kbctl status`
Web only	`kbctl dashboard open`
TUI only, on this box	`kbctl tui`
TUI over SSH, loopback	`kbctl ssh connect`
TUI over SSH, remote	`kbctl ssh connect HOST -i KEY`
Add an operator	`sudo kbctl ssh add-key KEY.pub`
Contain, then prove it	`kbctl process isolate ... && kbctl audit verify`
Policy edit, apply	`sudo $EDITOR /etc/kb/policy.yaml && sudo kbctl policy reload`
Evidence copy	`kbctl audit export --out FILE`
Stop for maintenance	`sudo kbctl down`
Stop one surface permanently	`kbctl dashboard down` / `sudo kbctl ssh down`

CHAPTER 12

KB-CHECKER: UNDER MAINTENANCE AND REPAIRS

STATUS: DOWN, BY DECISION

`kb-checker` is under maintenance and repairs and is kept down. This is deliberate and enforced in three places, so that no one starts it by accident.

12.1 How it is kept dead, and how you start it

Nothing starts `kb-checker` automatically. A manual start is deliberately *allowed*.

Mechanism	Effect
The package never enables it	**No `systemctl enable` for `kb-checker.service` anywhere in the packaging.**
`/usr/lib/kb/migrate-legacy.sh`	**On every install and upgrade it runs `systemctl disable kb-checker`, so a checker enabled by an older dev install no longer starts at boot. On a *fresh install* it also stops it; on an *upgrade* a running checker is left alone, so a manual start survives upgrades.**
`kbctl up`	**Starts `kbd`, `kb-sensor`, `kbopd`, `sshd@kb-operator` only.**
`kbctl status`	**Prints `UNDER MAINTENANCE AND REPAIRS (kept down)`; if you have started it by hand, `active, STARTED MANUALLY (under maintenance and repairs)`.**
`kbagents.service`	**No longer `Requires=` the checker, which would have started it.**

```
$ systemctl is-enabled kb-checker      # disabled: no boot autostart
$ sudo systemctl start kb-checker      # your decision; it now runs
$ kbctl status | tail -1
kb-checker.service           active, STARTED MANUALLY (under maintenance and repairs)
$ sudo systemctl stop kb-checker       # back to dead
```

12.2 What you lose while it is down

`kb-checker` audits eBPF bytecode and map integrity, sensor liveness, the `kbd` gRPC health, and the swarm. It also runs the *Tampering Containment Protocol*: on a confirmed integrity violation it stops the sensor, detaches the eBPF links and quarantines the host's network. With it down:

- No automatic tamper response. Nothing independently notices if the sensor or its programs are altered. Treat the host as *observed but not self-defending*.
- No false lockdowns either. The checker's fail-closed behaviour can isolate a host over a missing dependency (for example, an absent swarm). Its absence removes that risk during demos.
- `/api/services` in the dashboard will report the checker as down. That is expected, not a fault.

A MANUAL START ARMS A COMPONENT THAT CAN LOCK DOWN THE HOST

Starting it by hand can stop the sensor and firewall the host. Do it only on a machine you can reach out-of-band. Never `systemctl enable` it as a workaround; the next package install or upgrade removes that autostart again. Once the repairs are signed off, delete the `kb-checker.service` line from `maintenanceUnits` in `kb-op/kbctl/svc.go`, add the unit to `stackUnits`, drop the `systemctl disable kb-checker` step from `packaging/scripts/migrate-legacy.sh`, and rebuild.

12.3 Design invariants that still apply

The watchdog is intentionally stateless and opens no network listener (it serves diagnostics only on the local socket `/run/kb/kbc.sock`). When repairs resume, do not add persistent state or a network listener to it: that is a security design invariant of the project, not a style choice.

CHAPTER 13

DEMONSTRATION RUNBOOK: INSTALL TO UNINSTALL

This is the whole demonstration, in order, from an empty host to an empty host. Follow it top to bottom. Every step has the command, what you should see, and an honest status tag:

VERIFIED ON VM	the step was run on the Ubuntu 24.04 test VM and behaved as written
NOT YET RUN	the step has not been run; the expected output is derived from the code
FROM SOURCE, NOT RUN	as above, and the exact wording is taken from the source rather than a live run

Steps marked NOT YET RUN are the ones to rehearse before an audience. Section 13.16 shows how to run the install and checks automatically on the VM.

13.1 Time budget and layout

Phase	What	Screen / terminal
0–3	Prepare, install, verify (about 4 min)	Terminal A (the demo host)
4	Start the stack (1 min)	Terminal A
5	Sensor and control plane live (2 min)	Terminal B: `journalctl` follow
6	Web dashboard (2 min)	Browser
7	SSH console (2 min)	Terminal C
8	Contain a process (2 min)	Terminals A and C
9–10	Audit proof, policy reload (2 min)	Terminal A
11	Failure and recovery (1 min)	Terminals A and C
12–14	Stop, uninstall, reset (3 min)	Terminal A

Open three terminals on the demo host before you start (A: commands, B: logs, C: the SSH console) and one browser window.

13.2 Phase 0: Before you start

1 Confirm the host meets the prerequisites (Chapter 3)

```
$ ls /sys/kernel/btf/vmlinux            # must exist
$ cat /sys/kernel/security/lsm          # must end with: ...,bpf
$ id -nG | tr ' ' '\n' | grep -x -E 'sudo|wheel'   # you need sudo
```

VERIFIED ON VM (`lockdown,capability,landlock,yama,apparmor,bpf` and BTF present on the test VM.)

2 Make sure no earlier install is running

```
$ systemctl is-active kbd kb-sensor kbopd sshd@kb-operator kb-checker
```

All should say `inactive` (or `unknown` on a clean host). VERIFIED ON VM If a development install is running, either stop it by hand or let the package's migration step retire it (Section 6.2).

3 Check the package file

```
$ sha256sum -c SHA256SUMS --ignore-missing
kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst: OK
```

VERIFIED ON VM

13.3 Phase 1: Install

1 Install the package

```
$ sudo pacman -U kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
```

NOT YET RUN (not run on Arch; the post-install hook is the same script as on Ubuntu.) Expected tail:

```
created group: kb
created user: operator (member of kb)
generated SSH host key: /etc/kb/ssh/ssh_host_ed25519_key
created empty /etc/kb/authorized_keys - add operator public keys here
provisioning complete
kb: generated the dashboard API token in /etc/kb/api.env
==> Kernel Borderlands installed. Nothing has been started.
```

2 Prove nothing started and the checker is held down

```
$ kbctl status
kbd.service                  inactive
kb-sensor.service            inactive
kbopd.service                inactive
sshd@kb-operator.service     inactive
kb-checker.service           UNDER MAINTENANCE AND REPAIRS (kept down)
$ systemctl is-enabled kb-checker
disabled
```

VERIFIED ON VM

3 Check the provisioning

```
$ getent passwd operator
operator:x:998:994::/:/bin/sh
$ sudo stat -c '%a %U:%G' /etc/kb/api.env
640 root:kb
```

VERIFIED ON VM **(UID and GID numbers differ per host; the home is / and the shell `/bin/sh`.)**

13.4 Phase 2: Authorize an operator and start the stack

1 Authorize your SSH public key for the console

```
$ sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub
key authorized for operator
```

VERIFIED ON VM **(If you have no key: `ssh-keygen -t ed25519`.)**

2 Bring the stack up

```
$ sudo kbctl up
$ kbctl status
kbd.service                  active
kb-sensor.service            active
kbopd.service                active
sshd@kb-operator.service     active
kb-checker.service           UNDER MAINTENANCE AND REPAIRS (kept down)
```

VERIFIED ON VM

3 Check the sockets and ports

```
$ ls -l /run/kb
srw-rw---- 1 root kb   ... kba.sock       # group kb: the operator console can reach it
srwxr-xr-x 1 root root ... kbct.sock
srwxr-xr-x 1 root root ... kbd.sock
$ ss -ltn | grep -E ':(8080|8082|2222)\b'
```

VERIFIED ON VM **Three listeners:** `127.0.0.1:8080` **(**`kbd` **API),** `127.0.0.1:8082` **(dashboard),** `0.0.0.0:2222` **(operator** `sshd`**).**

13.5 Phase 3: Show the sensor and the control plane working

1 In Terminal B, follow the sensor and `kbd`

```
$ sudo journalctl -f -u kb-sensor -u kbd
```

You will see a stream of `[process_exec]`**,** `[process_exit]` **and** `[privilege_change]` **lines from the sensor.** VERIFIED ON VM

2 In Terminal A, show the live counters

```
$ kbctl stats
events/sec:       112.40
active processes: 238
```

FROM SOURCE, NOT RUN **(numbers will differ).**

3 Trigger a detection with a safe, real event

Reading a credential file is something the sensor always flags. Do it from Terminal A:

```
$ sudo cat /etc/shadow > /dev/null
```

In Terminal B you should see lines like these (observed during the VM test, produced by `sudo` **itself):**

```
kbd_sensor: [file_access] PID=28539 UID=1000 COMM=sudo file=/etc/shadow SENSITIVE
kbd_sensor: [BEHAVIOR ENGINE] PID=28539 COMM=sudo State transition: OBSERVED -> SUSPICIOUS (Reason:
    • CRED_FILE_ACCESS ...)
kbd: [KB] Zone PID=28555 SAFE->SUSPICIOUS score=9.8
kbd: [AUDIT] ZONE_SAFE_TO_SUSPICIOUS | pid=28555 ... score=9.81 ...
```

VERIFIED ON VM **Say what happened: the kernel hook saw the file open, the behavior engine moved the process to a higher state, and** `kbd` **recorded the zone change in the tamper-evident audit log. This is a *detection*, not a block: containment is an operator decision (Phase 6).**

WHY THERE IS NO ATTACK SCRIPT HERE

The repository's `scripts/attack-lab/` **README lists six attack simulations, but only** `containment_demo.py` **exists in the tree. This runbook therefore uses only harmless, real events: a credential-file read and a** `sleep` **process. Do not run attack simulations on a host that is not isolated.**

13.6 Phase 4: Web dashboard

1 Open it with the API token handed over

```
$ kbctl dashboard open
opened http://127.0.0.1:8082
```

VERIFIED ON VM **on the VM: the dashboard service is active and / returns 200.** NOT YET RUN **for** `kbctl dashboard open` **itself on the VM (it was tested locally against the packaged** `kbopd` **with a stub** `systemctl`**) and for the page content in a browser. The page loads at** `http://127.0.0.1:8082`**, the token is moved from the URL fragment into** `sessionStorage`**, and the address bar is cleaned. On a headless server, run** `kbctl dashboard url --token` **on the host and browse through an SSH tunnel (Chapter 8).**

2 Show the data and the action buttons

Point at the process table, the zone chart and the alert feed. Then show that the action buttons are authorized:

```
$ TOK=$(sudo grep ^KB_HTTP_API_TOKEN= /etc/kb/api.env | cut -d= -f2)
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST http://127.0.0.1:8080/api/policy/reload
    •                # no token
401
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST -H "Authorization: Bearer $TOK" http
    • ://127.0.0.1:8080/api/policy/reload
200
```

VERIFIED ON VM **(the 401 / 401 (wrong token) / 200 behaviour of the API). Clicking isolate and restore in the page is** NOT YET RUN**.**

13.7 Phase 5: The SSH console

1 In Terminal C, connect

```
$ kbctl ssh connect
```

VERIFIED ON VM **Key accepted, the terminal UI appears with four tabs (*Processes*, *Alerts*, *Agent Activity*, *Query Console*), live link to** `kbd` **(no** `[OFFLINE]` **banner), and it exits cleanly on** `q`**. From another machine:** `ssh -t -p 2222 operator@HOST`**.**

2 Drive it

`Tab` **through the views; on *Processes* press /** **and type** `sleep`**; on *Query Console* try** `help`**,** `stats`**,** `zone suspicious`**. The tab rendering is** VERIFIED ON VM**; the individual query commands are** NOT YET RUN**.**

3 **Show that it is a forced console, not a shell**

```
$ ssh -p 2222 operator@HOST id
```

You get the console, not the output of `id`. NOT YET RUN

13.8 Phase 6: Contain a process

1 **Start a harmless victim in Terminal A**

```
$ sleep 600 &
[1] 5120
```

2 **Contain it from the command line**

```
$ sudo kbctl process isolate --pid 5120 --level CGROUP --reason "demo"
pid=5120 contained at level CGROUP
```

FROM SOURCE, NOT RUN **The output format is taken from the source. Escalation order: `CGROUP`, `SECCOMP`, `NAMESPACE`, `TERMINATE`.**

3 **Show it in the console**

In Terminal C: *Processes*, filter `sleep`; the Containment column shows the level. Or contain from the TUI: select the row, press `c`, choose a level (`1`–`4`), confirm with `y`. NOT YET RUN

4 **Release and clean up**

```
$ kill 5120
```

13.9 Phase 7: The audit proof

1 **Verify the hash chain**

```
$ kbctl audit verify
chain intact - 1763 entries verified
```

VERIFIED ON VM **(the count differs per host). Every record hashes the one before it; one altered byte breaks the chain.**

2 **Export a copy**

```
$ kbctl audit export --out /tmp/demo-audit.json
wrote 1763 entries to /tmp/demo-audit.json
```

FROM SOURCE, NOT RUN

13.10 Phase 8: Change policy live

1 Edit and reload

```
$ sudo $EDITOR /etc/kb/policy.yaml      # e.g. lower bash's suspicious threshold
$ sudo kbctl policy reload
```

NOT YET RUN **for `kbctl policy reload` (gRPC). The same reload through the HTTP API with the token is** VERIFIED ON VM **(Phase 4).**

13.11 Phase 9: Failure and recovery

1 Stop the control plane under the console

```
$ sudo systemctl stop kbd
```

In Terminal C the TUI shows the `[OFFLINE] kbd unreachable` banner and fictional `[DEMO DATA]` rows. Point out that the banner is the honest signal and the rows are not live. NOT YET RUN **for stopping `kbd` under a live console. The banner itself was seen on the VM when the link to `kbd` failed (before the socket-group fix) and in unprivileged runs.**

2 Bring it back

```
$ sudo kbctl up
$ kbctl status
```

`kbctl up` after a stop is VERIFIED ON VM**. Whether an already-open console reconnects on its own is** NOT YET RUN**; if it does not, quit with `q` and run `kbctl ssh connect` again.**

13.12 Phase 10: Optional, the agent swarm

Only if the demo audience wants it, and only on a host where you have built the virtualenv (Chapter 15).

```
$ sudo systemctl start kbagents
```

```
$ curl -s http://127.0.0.1:8601/agents | head -c 300
```

Running the launcher by hand against an existing venv is VERIFIED ON VM (nine agents, zero errors); running it as the systemd unit is NOT YET RUN.

13.13 Phase 11: Stop everything

```
$ sudo kbctl down
$ kbctl status
kbd.service                 inactive
...
kb-checker.service          UNDER MAINTENANCE AND REPAIRS (kept down)
```

VERIFIED ON VM (`kbctl down` stops in reverse boot order.) To also stop them from starting at boot: `sudo kbctl dashboard down; sudo kbctl ssh down`.

13.14 Phase 12: Uninstall

1 Remove the package

```
$ sudo pacman -R kernel-borderlands
```

NOT YET RUN for the removal itself (the pre-removal hook disables and stops the units). The service-stopping part is the same logic as `kbuninstaller.sh`.

2 See what is deliberately left behind

```
$ ls /etc/kb /var/lib/kbd
$ getent passwd operator; getent group kb
$ ls /etc/systemd/system/*.pre-migration 2>/dev/null      # only if a dev install was migrated
```

These hold keys, the token, the audit chain and the account; a package manager should not destroy them.

3 Wipe everything (only if you want a truly clean host)

```
$ sudo rm -rf /etc/kb /var/lib/kbd /opt/kb
$ sudo rm -f /etc/systemd/system/*.pre-migration
$ sudo userdel operator && sudo groupdel kb
$ sudo systemctl daemon-reload
```

NOT YET RUN

4 Confirm

```
$ ss -ltn | grep -E ':(8080|8082|2222)\b' || echo "ports free"
$ command -v kbd kbctl || echo "binaries gone"
```

NOT YET RUN

13.15 Phase 13: Reset between demonstrations

To repeat the demonstration from nothing, either do Phase 12 and Phase 1 again, or use the two scripts (next section) which do the same on the Ubuntu VM.

13.16 Running install and checks automatically

The repository ships two scripts in `packaging/vm-test/`; they prompt for `sudo` once and must be run with a terminal.

```
$ ssh -t HOST 'bash ~/kbuninstaller.sh --remove-package'   # stop services, remove the package (
    • keeps data)
$ ssh -t HOST 'bash ~/kbinstaller.sh'                       # install, restart, check everything
...
=== SUMMARY ===
PASS=34 FAIL=0
```

VERIFIED ON VM **`kbinstaller.sh` checks Phases 1–5 and 7 (install, migration, token, CORS and auth, dashboard, services, the SSH console with a live `kbd` link, audit verify, sensor). It does *not* cover contain, policy reload, failure and recovery, the swarm, reboot persistence or uninstall; those are the NOT YET RUN rows above.**

13.17 The whole demonstration on one page

```
# --- install ---
sudo pacman -U kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
```

```
kbctl status                                  # all inactive, checker labelled
# --- start ---
sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub
sudo kbctl up && kbctl status
# --- show ---
sudo journalctl -f -u kb-sensor -u kbd        # terminal B
sudo cat /etc/shadow > /dev/null              # trigger a detection
kbctl dashboard open                          # browser
kbctl ssh connect                             # terminal C
```

```
sleep 600 & sudo kbctl process isolate --pid $! --level CGROUP --reason demo
kbctl audit verify
sudo kbctl policy reload
# --- recover ---
sudo systemctl stop kbd ; sudo kbctl up
# --- stop and remove ---
sudo kbctl down
```

```
sudo pacman -R kernel-borderlands
sudo rm -rf /etc/kb /var/lib/kbd ; sudo userdel operator ; sudo groupdel kb
```

PART IV

CONFIGURATION

CHAPTER 14

CONFIGURATION REFERENCE

14.1 Which file does what, and when it takes effect

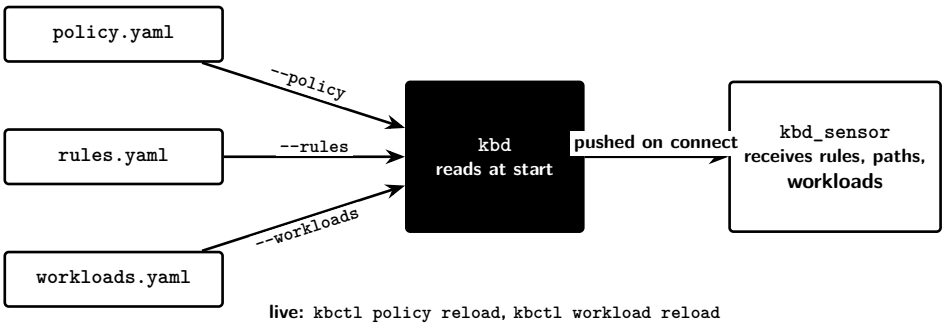


Figure 14.1: The config files are read by `kbd`, never by the sensor. The sensor only receives the compiled result when it connects.

File	Consumed by	Takes effect	Notes
`policy.yaml`	`kbd --policy`	`kbctl policy reload` (thresholds); `sensitive_paths` only on sensor reconnect	Per-process thresholds, extra watched paths.
`rules.yaml`	`kbd --rules`	On sensor (re)connect	Behavior-engine attack-chain rules, 19 by default.
`workloads.yaml`	`kbd --workloads`	`kbctl workload reload` (live)	Empty by default; CWP is opt-in.
`agents.yaml`	`kbagents --config`	On `kbagents` start	Swarm composition.
`allowlist.yaml` (repo only)	nothing today	–	Its own header says "Not yet consumed by the daemon". Not installed.
`kb.yaml` (repo only)	nothing today	–	No code references it; `kbd` has no `--config` flag. Not installed.
`api.env` (generated)	`kbd.service` via `EnvironmentFile`	`kbd` restart	Dashboard API token, `root:kb` 0640, created once at install, never overwritten (Section 8.5).
`sshd_kb-operator.conf`	operator `sshd`	`systemctl reload sshd@kb-operator`	Chapter 9.

DO NOT EDIT THE SENSOR'S COMPILED-IN FLOOR AWAY

Config can only *add* to what the sensor enforces (for example extra `sensitive_paths`); it can never narrow the compiled-in floor (`/etc/shadow`, `/etc/sudoers`, `/root/.ssh/`). That is intentional: a stolen config file must not be able to blind the sensor.

14.2 policy.yaml

Listing 14.1: kb-control-plane/config/policy.yaml

```
defaults:
  suspicious:   40.0
  borderlands:  75.0

# Additional paths tracked by the kernel LSM hook (lsm/file_open), on top
# of the compiled-in floor (/etc/shadow, /etc/sudoers, /root/.ssh/) in
# kb-core/userspace/sensor/kbd_sensor.c. Entries here are ADDITIVE ONLY -
# they can never remove or replace the compiled-in floor. Must be
# absolute paths, <= 63 bytes (fixed-size BPF map key), not bare "/".
# Takes effect on kbd_sensor (re)connect - not a live reload.
#
# NOTE: reads of these paths are always flagged for behavioral scoring,
# for every process. The hard -EACCES block only applies to a process
# already under operator containment at level >= 2 (Seccomp) - it is not
# a blanket block on normal system operation (sudo/PAM reading
# /etc/sudoers, /etc/shadow, etc. are unaffected unless already contained).
sensitive_paths:
  - /tmp/kb-pipeline-test-secret.txt

# auto_namespace_on_exfil_beacon (per-comm, default false): opt a process
# into automatic NAMESPACE containment when internal/detection's
# beaconing/exfiltration detector (docs/development/control-aads/
# dev-exfiltration-detection.md) flags it. Off by default - a statistical
# timing/frequency heuristic has real false-positive risk, so
# auto-enforcement stays an explicit per-comm choice, same posture as
# auto_terminate below.
policies:
  - comm: nginx
    suspicious:   55.0
    borderlands:  85.0
    allow_network: true
    auto_terminate: false

  - comm: postgres
    suspicious:   50.0
    borderlands:  80.0
    allow_network: false
    auto_terminate: true

  - comm: bash
    suspicious:   45.0
    borderlands:  70.0
    auto_terminate: false
```

Key	Type	Meaning
`defaults.suspicious`	float	Score at which a process enters the `SUSPICIOUS` zone (default 40).
`defaults.borderlands`	float	Score for `BORDERLANDS` (default 75). Must exceed `suspicious`.
`sensitive_paths[]`	absolute path	Extra files the LSM hook watches. $\leq$ 63 bytes, not bare /, additive only. Applied at sensor reconnect.
`policies[].comm`	string	Process name (the kernel `comm`, at most 15 characters) this entry applies to.
`policies[].suspicious / borderlands`	float	Per-process thresholds overriding the defaults.
`policies[].allow_network`	bool	Permits network activity for this `comm` without penalty.
`policies[].auto_terminate`	bool	Terminate automatically on `BORDERLANDS`. Dangerous; off by default.
`policies[].auto_namespace_on_exfil_beacon`	bool	Auto-contain at NAMESPACE when the beaconing detector flags it. Off by default: a timing heuristic has real false-positive risk.

THE ZONE LADDER

Scores move a process through `SAFE` $\rightarrow$ `SUSPICIOUS` (40) $\rightarrow$ `BORDERLANDS` (75). Scoring is computed in the sensor (EMA/`ScoreDelta`) and arrives pre-computed; `kbd` only applies these thresholds and forwards the results. Reads of watched paths are ***always flagged for scoring***; the hard `-EACCES` block applies only to a process an operator already contained at level $\geq$ 2 (Seccomp), so `sudo` and PAM keep working.

14.3 rules.yaml: the behavior engine

Each rule is a small state-machine edge: when the listed *event flags* occur, in the listed *order*, within `window_seconds`, a process currently at `min_source_state` or above moves to `target_state`.

Field	Meaning
`name`, `description`	Identifier and human text.
`required_flags`	Event flags that must all have been seen (`KB_EV_*`).
`optional_flags`, `optional_min`	At least `optional_min` of these must also be present.
`sequence`	Ordered steps (`KB_SEQ_*`) that must occur in this order.
`window_seconds`	Time window for the whole sequence.
`target_state`	State entered on a match: `KB_STATE_OBSERVED`, `SUSPICIOUS`, `BORDERLANDS`, `COMPROMISED`.
`reason`	Reason code recorded with the transition (`KB_REASON_*`).
`min_source_state`	Rule only fires for processes already at or above this state.

The 19 shipped rules, strongest effect first:

Target state	Rules
COMPROMISED (5)	known_ioc_sequence, reverse_shell_compromised, injection_ptrace_compromised, injection_proc_mem_compromised, escalation_exfil_compromised
BORDERLANDS	rwx_memory_abuse, ptrace_injection_borderlands, c2_port_connection, proc_mem_write_borderlands, rapid_connect_burst_borderlands
SUSPICIOUS	multi_anomaly_suspicious, privilege_plus_network_suspicious, cred_file_access_suspicious, anon_exec_mapping_suspicious, shell_spawn_suspicious
OBSERVED	unexpected_outbound_connection, privilege_change_observed, high_syscall_entropy_observed, first_anomaly_observed

The complete shipped file follows. Rules are compiled by `kbd` and sent to the sensor when it connects; if `--rules` is empty the sensor falls back to its compiled-in defaults.

Listing 14.2: kb-control-plane/config/rules.yaml

```
# Behavior Engine Detection Rules Policy.
# This file is loaded by the Go control plane and serialized to the C sensor over the Unix domain
    • socket.

rules:
  - name: known_ioc_sequence
    description: "Exact IOC sequence matched: exec -> outbound connect -> RWX memory"
    required_flags:
      - KB_EV_OUTBOUND_CONNECT
      - KB_EV_RWX_MAPPING
    optional_flags:
      - KB_EV_EXEC_FROM_TMP
      - KB_EV_EXEC_FROM_PROC
    optional_min: 1
    sequence:
      - KB_SEQ_EXEC
      - KB_SEQ_OUTBOUND_CONNECT
      - KB_SEQ_RWX_MAP
    window_seconds: 30
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_KNOWN_IOC_SEQUENCE
    min_source_state: KB_STATE_SAFE

  - name: reverse_shell_compromised
    description: "Outbound connection followed by shell execution"
    required_flags:
      - KB_EV_OUTBOUND_CONNECT
      - KB_EV_SPAWNED_SHELL
    sequence:
      - KB_SEQ_OUTBOUND_CONNECT
      - KB_SEQ_EXEC_SHELL
    window_seconds: 60
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_REVERSE_SHELL_CHAIN
    min_source_state: KB_STATE_BORDERLANDS

  - name: injection_ptrace_compromised
    description: "RWX memory allocation followed by ptrace injection"
    required_flags:
```

```
      - KB_EV_RWX_MAPPING
      - KB_EV_PTRACE_USED
    sequence:
      - KB_SEQ_RWX_MAP
      - KB_SEQ_PTRACE
    window_seconds: 30
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_INJECTION_CHAIN
    min_source_state: KB_STATE_BORDERLANDS

  - name: injection_proc_mem_compromised
    description: "RWX memory allocation followed by writing to proc memory"
    required_flags:
      - KB_EV_RWX_MAPPING
      - KB_EV_PROC_MEM_WRITE
    sequence:
      - KB_SEQ_RWX_MAP
      - KB_SEQ_PROC_MEM_WRITE
    window_seconds: 30
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_INJECTION_CHAIN
    min_source_state: KB_STATE_BORDERLANDS

  - name: escalation_exfil_compromised
    description: "Privilege escalation followed by credential file access and outbound connection"
    required_flags:
      - KB_EV_PRIVILEGE_GAINED
      - KB_EV_OUTBOUND_CONNECT
    optional_flags:
      - KB_EV_SHADOW_ACCESS
      - KB_EV_PASSWD_ACCESS
      - KB_EV_SSH_KEY_ACCESS
    optional_min: 1
    window_seconds: 120
    target_state: KB_STATE_COMPROMISED
    reason: KB_REASON_ESCALATION_EXFIL_CHAIN
    min_source_state: KB_STATE_BORDERLANDS

  - name: rwx_memory_abuse
    description: "RWX memory mapping or write to proc memory detected"
    optional_flags:
      - KB_EV_RWX_MAPPING
      - KB_EV_PROC_MEM_WRITE
      - KB_EV_PROCESS_VM_WRITE
    optional_min: 1
    target_state: KB_STATE_BORDERLANDS
    reason: KB_REASON_RWX_MEMORY
    min_source_state: KB_STATE_SUSPICIOUS

  - name: ptrace_injection_borderlands
    description: "Ptrace system call used by suspicious process"
    required_flags:
      - KB_EV_PTRACE_USED
    target_state: KB_STATE_BORDERLANDS
    reason: KB_REASON_PTRACE_INJECTION
    min_source_state: KB_STATE_SUSPICIOUS

  - name: c2_port_connection
```

```
    description: "Outbound connection to suspected C2 port"
    required_flags:
      - KB_EV_OUTBOUND_CONNECT
      - KB_EV_C2_CANDIDATE_PORT
    target_state: KB_STATE_BORDERLANDS
    reason: KB_REASON_C2_PORT_CONNECT
    min_source_state: KB_STATE_SUSPICIOUS

  - name: proc_mem_write_borderlands
    description: "Write to /proc/*/mem by suspicious process"
    required_flags:
      - KB_EV_PROC_MEM_WRITE
    target_state: KB_STATE_BORDERLANDS
    reason: KB_REASON_PROC_MEM_WRITE
    min_source_state: KB_STATE_SUSPICIOUS

  - name: rapid_connect_burst_borderlands
    description: "Rapid connections to multiple distinct destination IPs"
    required_flags:
      - KB_EV_RAPID_CONNECT_BURST
    target_state: KB_STATE_BORDERLANDS
    reason: KB_REASON_RAPID_CONNECT_BURST
    min_source_state: KB_STATE_SUSPICIOUS

  - name: multi_anomaly_suspicious
    description: "Multiple distinct behavioral anomaly categories set"
    optional_flags:
      - KB_EV_EXEC_FROM_TMP
      - KB_EV_EXEC_FROM_PROC
      - KB_EV_SPAWNED_SHELL
      - KB_EV_PRIVILEGE_GAINED
      - KB_EV_ROOT_ACHIEVED
      - KB_EV_RWX_MAPPING
      - KB_EV_ANON_EXEC
      - KB_EV_WX_TRANSITION
      - KB_EV_OUTBOUND_CONNECT
      - KB_EV_NONSTANDARD_PORT
      - KB_EV_SHADOW_ACCESS
      - KB_EV_SSH_KEY_ACCESS
      - KB_EV_HIGH_SYSCALL_ENTROPY
    optional_min: 3
    target_state: KB_STATE_SUSPICIOUS
    reason: KB_REASON_MULTI_ANOMALY
    min_source_state: KB_STATE_OBSERVED

  - name: privilege_plus_network_suspicious
    description: "Privilege change combined with outbound connection"
    required_flags:
      - KB_EV_PRIVILEGE_GAINED
      - KB_EV_OUTBOUND_CONNECT
    window_seconds: 60
    target_state: KB_STATE_SUSPICIOUS
    reason: KB_REASON_PRIVILEGE_PLUS_NETWORK
    min_source_state: KB_STATE_OBSERVED

  - name: cred_file_access_suspicious
    description: "Access to sensitive system credential stores"
    optional_flags:
```

```
      - KB_EV_SHADOW_ACCESS
      - KB_EV_PASSWD_ACCESS
      - KB_EV_SUDOERS_ACCESS
      - KB_EV_SSH_KEY_ACCESS
    optional_min: 1
    target_state: KB_STATE_SUSPICIOUS
    reason: KB_REASON_CRED_FILE_ACCESS
    min_source_state: KB_STATE_OBSERVED

  - name: anon_exec_mapping_suspicious
    description: "Anonymous executable memory mapping"
    required_flags:
      - KB_EV_ANON_EXEC
    target_state: KB_STATE_SUSPICIOUS
    reason: KB_REASON_ANON_EXEC_MAPPING
    min_source_state: KB_STATE_OBSERVED

  - name: shell_spawn_suspicious
    description: "Unexpected command interpreter invocation"
    required_flags:
      - KB_EV_SPAWNED_SHELL
    target_state: KB_STATE_SUSPICIOUS
    reason: KB_REASON_SHELL_SPAWN
    min_source_state: KB_STATE_OBSERVED

  - name: unexpected_outbound_connection
    description: "Any outbound connection"
    required_flags:
      - KB_EV_OUTBOUND_CONNECT
    target_state: KB_STATE_OBSERVED
    reason: KB_REASON_OUTBOUND_CONNECT
    min_source_state: KB_STATE_SAFE

  - name: privilege_change_observed
    description: "Credential change (UID drop / Root achieved)"
    optional_flags:
      - KB_EV_PRIVILEGE_GAINED
      - KB_EV_ROOT_ACHIEVED
      - KB_EV_CAP_GAINED
    optional_min: 1
    target_state: KB_STATE_OBSERVED
    reason: KB_REASON_PRIVILEGE_CHANGE
    min_source_state: KB_STATE_SAFE

  - name: high_syscall_entropy_observed
    description: "Divergent syscall profile (high syscall entropy)"
    required_flags:
      - KB_EV_HIGH_SYSCALL_ENTROPY
    target_state: KB_STATE_OBSERVED
    reason: KB_REASON_HIGH_SYSCALL_ENTROPY
    min_source_state: KB_STATE_SAFE

  - name: first_anomaly_observed
    description: "Any initial anomaly flag set"
    optional_flags:
      - 0xFFFFFFFFFFFFFFFF # Match any flag (this is handled as a special wildcard in parsing)
    optional_min: 1
    target_state: KB_STATE_OBSERVED
```

```
    reason: KB_REASON_FIRST_ANOMALY
    min_source_state: KB_STATE_SAFE
```

14.4 workloads.yaml: Critical Workload Protection

Listing 14.3: kb-control-plane/config/workloads.yaml

```
# CWP (Critical Workload Protection) protected-workload registry.
# See docs/features/CWP.md for the full design; this file follows S.12.1's
# YAML schema, trimmed to the fields actually sent over the wire today
# (path + identity_tier + expected_hash - owner_team/justification are
# parsed and logged for operator documentation value but not yet part of
# any alert, per CWP.md S.9's "still open" note).
#
# Loaded by kbd at startup (--workloads, default this file) and pushed to
# the sensor at connect time; `kbctl workload reload` re-reads this file
# and pushes the result live to every currently-connected sensor without
# restarting kbd.
#
# identity_tier: path (default) - matches on resolved executable path only.
# identity_tier: hash - also requires the binary's SHA-256 to match.
#   If expected_hash is omitted, kbd computes it locally from the file at
#   `path` on load (kbd runs on the same host as the sensor) - recompute by
#   re-running `kbctl workload reload` after every legitimate binary
#   upgrade, or the entry will start rejecting the new (correct) binary.
#
# Signing (optional, CWP.md S.4.3/S.11.4): if kbd is started with
# --workloads-pubkey <path>, this file MUST carry a valid policy_metadata
# block (version/signed_by/signature) or it is rejected outright - see
# `kbctl workload keygen` and `kbctl workload sign`. Unsigned (no
# policy_metadata.signature) is the default and stays valid as long as
# --workloads-pubkey is not set.
#
# Empty by default - CWP is opt-in. Uncomment/add entries for your own
# mission-critical applications.

critical_workloads: []

# Example:
# critical_workloads:
#   - path: /usr/bin/postgres
#     identity_tier: path
#     owner_team: data-platform
#     justification: "Primary production OLTP database"
#
#   - path: /usr/bin/vault
#     identity_tier: hash
#     owner_team: security-infra
#     justification: "Secrets management; elevated identity verification required"
```

Key	Meaning
`critical_workloads[].path`	Resolved executable path of the protected program.
`identity_tier: path`	(default) match on path only.
`identity_tier: hash`	Also require the binary's SHA-256. If `expected_hash` is omitted, `kbd` computes it from the file on load.
`owner_team`, `justification`	Documentation; parsed and logged, not yet part of any alert.

HASH TIER AND UPGRADES

After every *legitimate* upgrade of a hash-tier binary, run `sudo kbctl workload reload`, or the entry will start rejecting the new, correct binary. For signed registries, see Playbook 11.5.

14.5 agents.yaml

Listing 14.4: config/agents.yaml

```yaml
# Swarm composition for kb-aads. Loaded by kb-aads/main.py at startup.
#
# ray:
#   mode: local  -> ray.init() with no address, starts a single-node head
#                   in-process. Use for dev/single-node runs.
#   mode: cluster -> ray.init(address="auto"), joins an existing head node
#                    started separately via `ray start --head`. Use for
#                    the SOC/fleet deployment model (see
#                    docs/architecture/resource_management_roadmap.md).
ray:
  mode: local

# JUDGE/JURY/EXECUTOR are not counts like the others: one Judge and one
# Executor are spawned as long-lived actors; the Jury pool is spawned
# dynamically per consensus round (see docs/development/control-aads/
# ray-integration-walkthrough.md S.5B) and is not configured here.
# Containment militia squads are likewise not a count - a squad lead is
# spawned on demand per incident via RaySwarmOrchestrator.spawn_militia_squad
# (see agents/militia.py), not at swarm startup.
# Keys must match agents.base_agent.AgentRole values exactly (singular) -
# they're passed straight to AgentRole(role_name).
swarm:
  patroller: 2
  hunter: 2
  healer: 1
  containment: 1
  # signal_relay routes to the hunter pool today (see
  # agents/signal_relay.py's scoping note on which routes exist).
  signal_relay: 1

jury:
  pool_size: 5

```

```
# suspicious_threshold: score at which Patroller escalates a PID to
# Hunter (agents/patroller.py). Deliberately separate from
# kb-control-plane's config/policy.yaml `defaults.suspicious` (also 40.0
# by default) - the two systems can reasonably be tuned independently.
patroller:
  suspicious_threshold: 40.0

control_plane:
  grpc_socket: /run/kb/kba.sock
```

Key	Meaning
`ray.mode: local`	Start a single-node Ray head inside the process.
`ray.mode: cluster`	Join an existing head (`ray start --head`) for fleet deployments.
`swarm.*`	Actor counts: `patroller` 2, `hunter` 2, `healer` 1, `containment` 1, `signal_relay` 1. Keys must match the agent roles exactly (singular).
`jury.pool_size`	Jury size per consensus round (judge and executor are single long-lived actors).
`patroller.suspicious_threshold`	Score at which the Patroller escalates a PID to a Hunter. Tuned independently of `policy.yaml`.
`control_plane.grpc_socket`	`/run/kb/kba.sock`.

14.6 allowlist.yaml and kb.yaml

Listing 14.5: kb-control-plane/config/allowlist.yaml

```
# Process allowlist - comm names that are exempt from default scoring
# thresholds because they are known, trusted, and have unusual-but-legitimate
# behavioral signatures (e.g. init systems, package managers).
#
# Not yet consumed by the daemon - wire this into the Policy Engine
# alongside policy.yaml.

allow:
  - comm: systemd
    reason: "PID 1 / init system"
  - comm: sshd
    reason: "Trusted remote access daemon"
```

Listing 14.6: kb-control-plane/config/kb.yaml

```
# NOT CONSUMED: no code in kbd, kbd_sensor or kb-checker reads this file today and kbd has
# no --config flag (verified by search, 2026-10-27). These values are design intent only;
# the real settings are kbd's flags (--grpc-socket, --http-addr, --policy, ...) and
# policy.yaml's thresholds. It is not installed by the packages.
grpc_port: 50051

scoring:
  alpha: 0.3
  thresholds:
    suspicious: 40
```

```
    borderlands: 75

enforcement:
  mode: permissive  # permissive | enforcing

audit:
  path: /var/log/kb/audit.log
  remote_siem: ""
```

Neither is read by any shipped binary (confirmed by searching the daemon, sensor and checker sources; `config/dashboard.yaml` has no consumer either). Editing them would change nothing, so neither is installed, and both are marked `NOT CONSUMED` in the repository (`kb.yaml`'s header, `config/README.md`). `kb.yaml` (gRPC port, scoring alpha, enforcement mode, audit path, SIEM target) is design intent only; the real settings are `kbd`'s flags and `policy.yaml`.

14.7 Overriding without editing packaged files: systemd drop-ins

Never edit `/usr/lib/systemd/system/*.service`; the next upgrade overwrites it. Use a drop-in, which `systemctl edit` creates under `/etc/systemd/system/<unit>.d/override.conf`.

```
$ sudo systemctl edit kbd
# add:
[Service]
Environment=KB_HTTP_CORS_ORIGIN=http://127.0.0.1:8082,http://localhost:8082,http://kb-host:8082
$ sudo systemctl restart kbd
```

THE API TOKEN IS NOT SET HERE

The dashboard token is generated for you into `/etc/kb/api.env` and loaded by the packaged unit (Section 8.5); do not put it in a drop-in.

NOTE

Repeating an `Environment=` line in a drop-in *adds* a variable; setting the same name again overrides it. `KB_HTTP_CORS_ORIGIN` is one comma-separated value, so restate the whole list when you extend it. To replace `ExecStart` you must first clear it with an empty `ExecStart=` line (as in the CWP playbook).

Inspect the merged result any time with `systemctl cat kbd` and `systemctl show kbd -p Environment`.

CHAPTER 15

KB-AADS: THE AGENT SWARM (OPTIONAL)

kb-aads is a Python/Ray multi-agent system that consumes kbd's data and coordinates response. It has no compiled build. The package installs its sources under /usr/lib/kb/aads/ and a launcher, kbagents; it does not install Ray or PyTorch, which are several gigabytes and CPU-specific (the development virtualenv is 5.4 GB). The unit is installed but not enabled.

STATUS: RUN AND VERIFIED, WITHOUT SYSTEMD

The packaged launcher and sources were run on the Ubuntu 24.04 VM with that machine's existing virtualenv (Ray 2.56): Ray started a local instance, the status server listened on 127.0.0.1:8601, and GET /agents returned nine agents (executor, judge, two hunters, two patrollers, and more), all active with error_count 0; the containment agent loaded the shipped checkpoint. The same launcher also reached the swarm banner on Arch with a scratch Python 3.11 venv (Ray 2.59, CPU torch), where Ray's own startup then stalled on a networking problem of that machine. Not tested: running it as kbagents.service under systemd, with a live kbd behind it.

15.1 Create the environment

1 Install Python tooling

```
$ sudo pacman -S --needed python python-pip
```

2 Build the venv where the launcher looks for it

```
$ sudo python3 -m venv /opt/kb/venv
$ sudo /opt/kb/venv/bin/pip install -r /usr/lib/kb/aads/requirements.txt
```

Requirements: ray[rllib,default]>=2.9.0, grpcio>=1.50.0, grpcio-tools, pyyaml, gymnasium, torch>=2.0.0, plus pytest for tests. Expect a long download and several GB of disk.

3 Check the launcher is satisfied

```
$ kbagents --help
usage: main.py [-h] [--config CONFIG]
```

Before the venv exists, `kbagents` exits 1 and prints exactly the `pip install` command to run.

15.2 Run it

To try it without installing the package, point the launcher at an unpacked tree and an existing venv:

```
$ KB_AADS_DIR=/path/to/usr/lib/kb/aads KB_AADS_PYTHON=/path/to/venv/bin/python3 \
  KB_AADS_CONFIG=/path/to/agents.yaml RAY_AUTH_MODE=disabled kbagents
$ curl -s http://127.0.0.1:8601/agents
```

The status server answers on `/agents` (the bare `/` returns nothing). `RAY_AUTH_MODE=disabled` stops newer Ray versions from generating a token file in your home directory; leave it unset in the service. To run it for real:

```
$ sudo systemctl enable --now kbagents
$ systemctl status kbagents
$ curl -s http://127.0.0.1:8601/agents   # swarm status server
```

`kbagents.service` runs `/usr/bin/kbagents`, which `cd`s to `/usr/lib/kb/aads` and executes `main.py --config /etc/kb/agents.yaml`. It `Requires=kbd.service`; it deliberately does not require the maintenance-held `kb-checker`.

15.3 How `kbd` finds it

`kbd` proxies the swarm's status for the dashboard's `/api/agents` route. The status server binds `127.0.0.1:8601` by default (`KB_AADS_STATUS_HOST`, `KB_AADS_STATUS_PORT`); `kbd`'s `KB_AADS_API_ADDR` must point at the same address if you change it.

KNOWN GAP

The event contract notes that nothing in `kb-aads` yet *consumes* `kbd`'s `process_state` / `zone_transition` events. The swarm is wired and runs, but event-driven behaviour is still being built out. Do not present it as autonomous response.

15.4 Hardening in the unit

`NoNewPrivileges`, `PrivateTmp`, `ProtectHome`, `MemoryDenyWriteExecute`, `ProtectSystem=strict`, an empty capability set, and read-only access to `/etc/kb`, `/usr/lib/kb`, `/opt/kb`; only `/run/kb` is writable. If Ray or PyTorch turn out to need JIT memory (`MemoryDenyWriteExecute` blocks it), relax that single directive in a drop-in and document why.

PART V

REFERENCE

CHAPTER 16

TROUBLESHOOTING

16.1 First five commands, always

```
$ kbctl status
$ systemctl --failed
$ journalctl -u kbd -u kb-sensor -u kbopd -u sshd@kb-operator -b --no-pager | tail -60
$ ls -l /run/kb/
$ ss -ltnp | grep -E ':(8080|8082|2222|8601)\b'
```

16.2 Symptom table

Symptom	Cause and fix
`kbd` sits in `activating (start)` forever; sensor says "Dependency failed"	`kbd.service` is `Type=notify` and never got `READY=1`. Fixed in this release (kbd notifies after its sockets exist). If you see it, you are running an old `kbd`.
`unable to open database file` crash loop	`/var/lib/kbd` missing. The unit now sets `StateDirectory=kbd`; an old unit does not.
`dial unix /run/kb/kba.sock: no such file` from `kbctl`	`kbd` is not running (`/run/kb` is removed when it stops). `sudo kbctl up`.
`sshd@kb-operator` loops: `Bind to port 2222 ... Address already in use`	The operator config sits in `/etc/ssh/sshd_config.d/`, so the *host* `sshd` bound 2222. Installing or upgrading the package now repairs this automatically (Section 6.2). By hand: move it to `/etc/kb/sshd_kb-operator.conf`, point the unit there, reload the host `sshd`, restart the unit.
`kbctl ssh connect: Permission denied (publickey)`	Your key is not in `/etc/kb/authorized_keys`: `sudo kbctl ssh add-key KEY.pub`. Check `-i`.
Journal shows `User operator not allowed because account is locked`; the client falls back to a password prompt	`useradd` leaves the shadow password as `!` (locked) and, with `UsePAM no`, OpenSSH refuses a locked account even for key logins. Current packages set it to `*` (no usable password, not locked); by hand: `sudo usermod -p '*' operator`.
Connection drops before login; journal shows `chroot("/run/sshd"): Operation not permitted [preauth]`	The operator `sshd` unit's empty `CapabilityBoundingSet=` blocks sshd's privilege separation. Fixed in the unit (it now grants `CAP_SYS_CHROOT CAP_SETUID CAP_SETGID CAP_CHOWN CAP_KILL CAP_AUDIT_WRITE`); reinstall the current package.
Connected, then "This account is currently not available."	the `operator` account still has a `nologin` shell, which refuses the `ForceCommand`. Current packages fix this on install; by hand: `sudo usermod --shell /bin/sh operator` (Section 9.4).
TUI shows `[OFFLINE]` and `[DEMO DATA]`	`kbd` unreachable. Fictional rows. Either `kbd` is not running, or the `operator` account cannot open `/run/kb/kba.sock` (older packages left it `root:root`; check `ls -l /run/kb/kba.sock` shows group `kb`, and that `kbd` was restarted on the current unit).
Dashboard loads but panels are empty	CORS: page origin not in `KB_HTTP_CORS_ORIGIN` (Chapter 8).
Dashboard buttons for isolate/restore do nothing	`KB_HTTP_API_TOKEN` not set; routes fail closed.
`kbopd` panics "Could not initialize background MCP sub-process"	`mcp-server` not found next to `kbopd`, in `/usr/bin` or in `/usr/local/bin`. Reinstall the package.
Package install does nothing new; old behaviour persists	A development install shadows the package: units in `/etc/systemd/system` override `/usr/lib/systemd/system`; `/usr/local/bin` precedes `/usr/bin`.
Sensor starts but file blocks never happen	BPF LSM not active: `cat /sys/kernel/security/lsm` must end with `bpf` (Chapter 3).
`kb-checker` will not start	By design (maintenance hold). See Chapter 12.
`kbagents` exits 1 "missing Python dependencies"	Build `/opt/kb/venv` (Chapter 15).

16.3 Build-time failures

Error	Cause and fix
`fatal error: 'bpf/libbpf.h' file not found`	A non-system `clang` (Nix, etc.) earlier in `PATH`. Build with `PATH=/usr/local/bin:/usr/bin`; install `libbpf` headers.
`undefined symbol: ring_core_0_17_14_...`	`makepkg` LTO. Use `options=(!lto)` (already in the PKGBUILD).
`undefined symbol: bpf_prog_get_next_id` (linking `kb-checker`)	makepkg's exported `LDFLAGS`/`RUSTFLAGS`. Build with them unset (the PKGBUILD does).
`tsc: not found`	Dashboard dependencies missing; the Makefile now runs `npm ci` when `node_modules` is absent. Needs network once.
`folder ... kb-dashboard/dist/ does not exist` (Rust)	Dashboard not built before `cargo`. Run the full `make build`.
`cargo: command not found` over non-login SSH	Export `PATH=$HOME/.cargo/bin:...`.
`dpkg-deb: control directory has bad permissions 2755`	Setgid checkout. `build-deb.sh` clears it; update the script.
`GLIBC_2.4x not found` on the target	Built on a newer distro. Rebuild on the target (or older).

16.4 Where the logs are

What	Where
All unit output	`journalctl -u UNIT` (stdout/stderr go to the journal).
Audit log	`kbd`'s hash-chained audit store; read it with `kbctl audit export` and `kbctl audit verify` (the on-disk location is internal to `kbd`).
Operator SSH sessions	Journal of `sshd@kb-operator`; start/end events in the audit log via the wrapper.
Config in effect	`systemctl cat UNIT`, `systemctl show UNIT -p Environment,ExecStart`.

16.5 Documentation and implementation disagreements

The repository's `docs/` directory is canonical. Two differences between it and what the packages ship were found and resolved by correcting the documentation (`boot_sequence_spec.md`, 2026-10-27):

- The operator `sshd` config lives at `/etc/kb/sshd_kb-operator.conf`, not `/etc/ssh/sshd_config.d/`. The spec, the source unit and `install.sh` now agree (see the port-2222 row

above).

- `kb-checker` is part of the designed boot but is held down by decision; the spec now carries a maintenance notice in Phase 2.

One item remains open by nature: `kb.yaml` and `allowlist.yaml` describe settings no code reads (Chapter 14). They are marked as such and are not installed.

CHAPTER 17

ACCEPTANCE CHECKLIST

The automated version of most of this is `packaging/vm-test/kbinstaller.sh` (Chapter 13, Section 13.16); the demonstration itself is Chapter 13. Run this top to bottom on the *target* host after installing the package. Each row has the command, the result that means PASS, and a box to tick. Rows marked ★ close a claim that this release's build sessions could not verify (no root); do not skip them before a demonstration.

17.1 A. Pre-flight

	Command	PASS looks like
☐	`ls /usr/local/bin/kb* /etc/systemd/system/kb*`	"No such file" (no development install shadowing the package)
☐	`ls /sys/kernel/btf/vmlinux`	file exists
☐	`cat /sys/kernel/security/lsm`	ends with `bpf`
☐	`sha256sum kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst`	matches the release's recorded hash

17.2 B. Install

	Command	PASS looks like
☐★	`sudo pacman -U kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst`	**Completes; prints "provisioning complete". (a real install is untested)**
☐	`getent passwd operator; getent group kb`	**both exist**
☐	`sudo ls -l /etc/kb/ssh/ssh_host_ed25519_key`	**exists, root-readable only**
☐	`systemctl cat kbd | grep CORS`	**shows the 8082 origins**
☐	`kbctl status`	**four `inactive`, checker line says UNDER MAINTENANCE AND REPAIRS**

17.3 C. Start and the sensor

	Command	PASS looks like
☐★	`sudo kbctl up`	**returns 0 with no unit failures. (boot under real systemd is untested)**
☐	`kbctl status`	**four `active`**
☐	`ls /run/kb`	`kbd.sock kbct.sock kba.sock` **(no `kbc.sock`: only the held-down checker creates it)**
☐★	`journalctl -u kb-sensor -b | grep -i -E 'attach|hook|lsm'`	**hooks attached; no LSM failure. (sensor under the package is untested)**
☐	`journalctl -u kbd -b | grep 'hash chain'`	**"verified intact"**
☐	`kbctl stats`	**prints events/sec and active processes**

17.4 D. Dashboard and API

	Command	PASS looks like
☐	`kbctl dashboard status; echo $?`	`active ... listening=true`, exit 0
☐★	`curl -si -H 'Origin: http://127.0.0.1:8082' http://127.0.0.1:8080/api/services | grep -i access-control-allow-origin`	header present and equal to the origin. (CORS runtime path is source-verified only)
☐	same request with `Origin: http://evil.example`	*no* `Access-Control-Allow-Origin` header
☐	`kbctl dashboard open`	browser opens; panels fill; no red CORS errors in the browser console
☐	`sudo stat -c '%a %U:%G' /etc/kb/api.env`	`640 root:kb`
☐★	in the dashboard, isolate a harmless process (`sleep 600`) and click restore	both succeed; the page's address bar shows no `#token=` afterwards. (untested in a browser)
☐	reload the page *without* the fragment (type the bare URL in a new tab) and press isolate	reports "API token missing or wrong"

17.5 E. SSH console

	Command	PASS looks like
☐	`sudo kbctl ssh add-key ~/.ssh/id_ed25519.pub`	"key authorized for operator"
☐	`kbctl ssh status; echo $?`	active, listening, exit 0 (no `Address already in use` in `journalctl -u sshd@kb-operator`)
☐★	`kbctl ssh connect`	the TUI appears. (verified on the Ubuntu VM; confirm on your host) If refused: Section 9.4
☐	in the TUI: `Tab` through four tabs, `q`	clean exit, shell returns
☐	`ssh -p 2222 operator@HOST id` from another machine	does not run `id`; shows the TUI (ForceCommand wins)
☐	`ssh -p 2222 -L 9999:127.0.0.1:8080 operator@HOST`	forwarding refused (`AllowTcpForwarding no`)

17.6 F. Containment and audit

```
$ sleep 600 &                        # a harmless victim
[1] 5120
$ sudo kbctl process isolate --pid 5120 --level CGROUP --reason "acceptance test"
pid=5120 contained at level CGROUP
$ kbctl audit verify
chain intact - N entries verified
$ kbctl audit export --out /tmp/acceptance-audit.json
$ kill 5120
```

	PASS looks like
☐	the isolate line prints and the TUI Processes tab shows CGROUP for that PID
☐	`audit verify` reports intact; the export contains your reason string

17.7 G. Maintenance hold

	Command	PASS looks like
☐	`kbctl status | tail -1`	UNDER MAINTENANCE AND REPAIRS (kept down)
☐	`systemctl is-active kb-checker`	`inactive` after install, after `kbctl up`, and after a reboot
☐	`systemctl is-enabled kb-checker`	`disabled` (no boot autostart)

17.8 H. Reboot and removal

	Command	PASS looks like
☐★	`sudo reboot`, then `kbctl status`	the four units came back on their own (`up` enabled them)
☐	`sudo pacman -R kernel-borderlands`	units disabled; `/etc/kb/ssh` and `/var/lib/kbd` retained

Record any failed row with the exact command, its output, and `journalctl -b` for the unit, and file it against the packaging (`packaging/`) rather than patching the host by hand.

APPENDIX A

WIRE AND EVENT CONTRACTS

Two independent contracts exist; confusing them is the most common mistake.

A.1 C to Go: `kbd.sock`, hand-packed structs

Packed little-endian structs, byte-identical in C (`__attribute__((packed))`) and Go.

Struct	Size
`ProcessState` / `kb_wire_process_state`	128 bytes
`ZoneTransition` / `kb_wire_zone_transition`	40 bytes
`kb_wire_attack_rule`	220 bytes

None carries an `event_type` field. The sensor's own `kb_unified_event.event_type` enum (`KB_EVT_PROCESS_EXEC`=0 through `KB_EVT_DROPPED_TELEMETRY`=10) never crosses the wire as a named field. Sources of truth: `kb-core/userspace/bridge/kb_bridge.h`, `docs/development/core-control/wire-protocol.md`.

WARNING

Change a wire struct in C and Go *together*, and verify against `docs/architecture/kbd-contracts.md`. A one-byte mismatch silently corrupts every later field.

A.2 Go to clients: `kba.sock`, `pb.KBEvent`

Exactly two live `event_type` strings:

event_type	Metadata keys
`process_state`	`zone`, `composite`, `dim_syscall`, `dim_privilege`, `uid`
`zone_transition`	`from_zone`, `to_zone`

Both carry `Pid`, `Comm`, `ScoreDelta`, `Timestamp` (and `Ppid` for state). Alerts are a separate

`pb.Alert` **stream, currently only** `BORDERLANDS_ENTRY`**. Scoring is entirely the sensor's job;** `kbd` **forwards the pre-computed** `ScoreDelta`**.**

APPENDIX B

KERNEL HOOK POINTS

Hook	Watches
`tracepoint:syscalls`	Every syscall entry and exit.
`tracepoint:sched`	fork, exec, exit.
`kprobe:commit_creds`	Privilege (credential) changes.
`bpf_lsm` (`file_open`)	Sensitive file access; the only hook that can block.
`tracepoint:net`	Network connections.
`kprobe:mmap_region`	Executable memory mappings.
uprobes on `libssl`, `libgnutls`, Go TLS	Plaintext of TLS writes.

Events travel to userland over a ring buffer. The roadmap in `docs/architecture/hookpoints.md` proposes further hooks (`process_vm_writev`, `security_capable`, `do_execveat_common`, writes to `/proc/*/mem`); they are *proposals*, not shipped.

APPENDIX C

CONDENSED CROSS-DISTRIBUTION RECIPE

C.1 Build the Arch package

```
arch$ sudo pacman -S --needed base-devel go rust clang bpf libbpf nodejs npm libelf zlib
arch$ cd packaging/arch && makepkg -f
arch$ sudo pacman -U kernel-borderlands-0.1.0-1-x86_64.pkg.tar.zst
```

C.2 Build the Ubuntu package

```
ubuntu$ sudo apt install -y build-essential clang llvm libelf-dev zlib1g-dev libbpf-dev \
          nodejs npm golang-go linux-tools-common linux-tools-$(uname -r)
ubuntu$ curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh && . ~/.cargo/env
ubuntu$ make build && packaging/deb/build-deb.sh
ubuntu$ sudo apt install ./dist/kernel-borderlands_0.1.0_amd64.deb
```

C.3 Differences in one table

Item	Arch	Ubuntu 24.04
Package tool	`makepkg` / `pacman -U`	`dpkg-deb` / `apt install ./`
Go	system `go` (new enough)	`golang-go` 1.22 bootstraps 1.25 via toolchain download
Rust	`rust` package	`rustup`
bpftool	package `bpf`	`linux-tools-$(uname -r)`
Boot param tool	`grub-mkconfig -o /boot/grub/grub.cfg`	`update-grub`
Config preservation	`backup=()` → `.pacnew`	`conffiles` → prompt
glibc of the build host	2.44	2.39

APPENDIX D

COMPLETE SYSTEMD UNIT FILES

These are the *source* units from `packaging/systemd/`. The packages install them with `/usr/local/bin` rewritten to `/usr/bin` and the `sshd` config path rewritten to `/etc/kb/sshd_kb-operator.conf`. Comments are the maintainers' own reasoning and are worth reading.

D.1 kbd.service

```
[Unit]
Description=Kernel Borderlands Go Control Plane Daemon
Before=kb-checker.service
DefaultDependencies=no
After=local-fs.target

[Service]
Type=notify
# Corrected from docs/architecture/boot_sequence_spec.md's `--config
# /etc/kb/config.yaml` - kbd (kb-control-plane/cmd/kbd/main.go) has no
# --config flag; it takes --db/--policy/--rules/--workloads separately.
# Pointed at /etc/kb/ so the provisioning story matches kb-checker's own
# real /etc/kb usage (kb-checker/src/integrity/mod.rs:21).
WorkingDirectory=/
# The packaged dashboard is served by kbopd on 127.0.0.1:8082 and its browser
# code fetches kbd's API on :8080 cross-origin. kbd's CORS allowlist defaults
# to the Vite dev server (:5173) only (internal/controlplane/http.go), so
# without this the dashboard loads but every API call is blocked by the
# browser. Mutating routes (isolate/restore) additionally need
# KB_HTTP_API_TOKEN set; they fail closed without it.
Environment=KB_HTTP_CORS_ORIGIN=http://127.0.0.1:8082,http://localhost:8082
# Runtime API token for the mutating routes (isolate/restore/policy-reload), generated at
# install by migrate-legacy.sh into /etc/kb/api.env (KB_HTTP_API_TOKEN=..., root:kb 0640).
# The leading '-' keeps kbd startable without it (the routes then fail closed).
EnvironmentFile=-/etc/kb/api.env
# kbd creates kba.sock as root:root 0660, so the `operator` account (group kb) could not
# reach it: the SSH TUI showed [OFFLINE] and the audit callback failed (found on a real
# host). Give ONLY the gRPC socket to group kb once kbd has signalled ready (Type=notify, so
# the socket exists). kbd.sock/kbct.sock stay root-only. '-' tolerates a missing kb group.
ExecStartPost=-/usr/bin/chgrp kb /run/kb/kba.sock
ExecStart=/usr/local/bin/kbd --db /var/lib/kbd/state.db --policy /etc/kb/policy.yaml --rules /etc/kb
    • /rules.yaml --workloads /etc/kb/workloads.yaml
```

```
Restart=always
RestartSec=3
RuntimeDirectory=kb
RuntimeDirectoryMode=0775
# Found by actually booting this unit in a container (not just
# systemd-analyze verify, which never runs ExecStart) - without this, kbd
# crash-loops immediately: "unable to open database file: no such file or
# directory", because /var/lib/kbd/ never gets created by anything.
# StateDirectory= is systemd's answer to exactly this - same mechanism as
# RuntimeDirectory=kb above for /run/kb, but persistent across reboots
# and owned by root here since no User= is set on this unit.
StateDirectory=kbd

# Conservative hardening only (no CapabilityBoundingSet=/ProtectSystem=strict
# - see kb-checker.service below for why): kbd is the Go control plane, no
# BPF/LSM loading, no capability-dropping code exists in it today, so this
# stays limited to directives that are universally safe regardless.
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
MemoryDenyWriteExecute=true

[Install]
WantedBy=multi-user.target
```

D.2 kb-sensor.service

```
[Unit]
Description=Kernel Borderlands eBPF Sensor (kbd_sensor) - Ring 0 Telemetry Loader
# NEW in this pass - docs/architecture/boot_sequence_spec.md never defined a
# unit for kbd_sensor itself, even though its own S.5 Scenario A already
# refers to stopping "kb-sensor.service" during the Tampering Containment
# Protocol. Source inspection (kb-control-plane/internal/ipc/listener.go:134,
# kb-core/userspace/bridge/kb_bridge.c:83) confirms kbd_sensor (C, eBPF) and
# kbd (Go) are two fully independent OS processes - kbd never exec/spawns
# kbd_sensor, it just binds kbd.sock first so kbd_sensor has something to
# connect to. This unit is what actually loads the eBPF hooks into the
# kernel; without it, "the boot sequence" never runs the sensor at all.
After=kbd.service
Requires=kbd.service
DefaultDependencies=no

[Service]
Type=simple
ExecStart=/usr/local/bin/kbd_sensor
Restart=always
RestartSec=3

# No hardening beyond the universally-safe directives below: loading eBPF
# programs and attaching LSM hooks needs CAP_BPF/CAP_PERFMON/CAP_SYS_ADMIN
# (exact set is kernel-version dependent - see
# docs/architecture/cross-kernel-portability.md), so this runs as root,
```

```
# consistent with kb-core/README.md's `sudo ./build/kbd_sensor`. Revisit
# CapabilityBoundingSet= only alongside an actual capability audit of
# kb-core's loader code, not as a systemd-only change.
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true

[Install]
WantedBy=multi-user.target
```

D.3 kb-checker.service (held down)

```
[Unit]
# UNDER MAINTENANCE AND REPAIRS: dead unless started by hand. Nothing starts it
# automatically: the packages never enable it, `kbctl up` skips it, kbagents.service
# does not require it, and packaging/scripts/migrate-legacy.sh disables any boot
# autostart. `systemctl start kb-checker` DOES start it, on purpose. Beware: its
# Tampering Containment Protocol can stop the sensor and firewall the host.
Description=Kernel Borderlands Safety and Integrity Watchdog Daemon
# kbd.service: Requires= is correct here - kb-checker never stops kbd
# itself, so if kbd genuinely goes down, kb-checker going down with it is
# the desired behavior (matches the original spec's design).
#
# kb-sensor.service: deliberately Wants=, NOT Requires=, even though the
# checker's Phase 2 boot audits (eBPF bytecode integrity, hook liveness)
# are meaningless before the sensor has loaded its hooks - see
# docs/architecture/boot_sequence_spec.md S.2 Phase 2 step 4. Found by
# actually booting this unit: Requires= means systemd stops kb-checker
# whenever kb-sensor stops, in EITHER direction - including when
# kb-checker's OWN recovery logic (report/mod.rs's trigger_auto_recovery)
# calls `systemctl stop/restart kb-sensor` as part of its normal
# operation. That self-inflicted stop-propagation killed kb-checker's own
# process (and its still-running `systemctl stop kb-sensor` child, via
# the whole cgroup receiving SIGTERM) in the middle of executing its own
# containment response - which then made a perfectly normal SIGTERM look
# like a failed stop, triggering the full pkill/bpf-detach/iptables
# Lockdown fallback from a daemon that was already being torn down.
# Wants=+After= keeps the same startup ordering (kb-checker only starts
# once kb-sensor has been asked to start) without the destructive stop
# propagation back onto kb-checker when IT is the one stopping the sensor.
After=kbd.service kb-sensor.service
Requires=kbd.service
Wants=kb-sensor.service
DefaultDependencies=no

[Service]
Type=simple
ExecStart=/usr/local/bin/kb-checker monitor --all --grpc-socket /run/kb/kbc.sock
Restart=always
RestartSec=5
PIDFile=/run/kb/kb-checker.pid
```

```
# Conservative hardening only - deliberately NO CapabilityBoundingSet= and
# NO ProtectSystem=strict here. kb-checker's Tampering Containment Protocol
# (Scenario A, S.4) shells out to `pkill -9`, `rm -rf /sys/fs/bpf/*`, and
# `iptables -P ... DROP` as a last-resort lockdown when tampering is
# actually detected - that needs root/CAP_KILL/CAP_NET_ADMIN/bpffs-delete
# rights, none of which are dropped anywhere in kb-checker's own code today.
# That path is only ever exercised during a real attack, not by routine
# tests, so getting a capability enumeration wrong here would fail exactly
# when it matters most and go unnoticed until then. Revisit only alongside
# actual privilege-dropping code changes in kb-checker itself, not as a
# systemd-only change.
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
MemoryDenyWriteExecute=true

[Install]
WantedBy=multi-user.target
```

D.4 kbopd.service

```
[Unit]
Description=Kernel Borderlands Operator Dashboard
After=kbd.service
Requires=kbd.service
DefaultDependencies=no

[Service]
Type=simple
# REAL as of this pass: kb-op/kb-tui was extended (axum + rust-embed) into
# a unified binary - the same compiled kb-tui binary, installed here as
# kbopd, embeds kb-dashboard's production build (npm run build's dist/)
# and serves it over HTTP when run without a controlling TTY (see
# kb-op/kb-tui/src/main.rs's is_terminal() check + run_headless(), and
# src/dash.rs for the embedded-asset server). The Makefile's `build`/
# `install` targets copy the compiled kb-tui binary to bin/kbopd and
# install it as /usr/local/bin/kbopd.
#
# ExecStart was previously /opt/kb/bin/kbopd --config /etc/kb/config.yaml
# - both wrong: /opt/kb/bin/ was a stale manual-test location, not where
# `make install` actually puts anything (/usr/local/bin/, matching every
# other unit in this directory), and kbopd parses no CLI arguments at all
# today (no clap/env::args() anywhere in main.rs), so --config was silently
# ignored. Running the stale /opt/kb/bin/kbopd directly reproduced a real
# crash (`Error: Os { code: 6, ... "No such device or address" }`) that
# the current, correctly-installed /usr/local/bin/kbopd does not have -
# confirmed by running both binaries directly, side by side.
ExecStart=/usr/local/bin/kbopd
Restart=always
RestartSec=5

# Full hardening: kbopd's real runtime need is a network listener
```

```
# (embedded dashboard server, src/dash.rs, hardcoded to 127.0.0.1:8082)
# plus spawning /usr/local/bin/mcp-server as a child - nothing privileged.
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
MemoryDenyWriteExecute=true
ProtectSystem=strict
ReadWritePaths=/run/kb
CapabilityBoundingSet=

[Install]
WantedBy=multi-user.target
```

D.5 sshd@kb-operator.service

```
[Unit]
Description=Kernel Borderlands SSH Operator Console (dedicated sshd instance, port 2222)
After=network.target
DefaultDependencies=no

[Service]
Type=notify
ExecStart=/usr/sbin/sshd -D -f /etc/kb/sshd_kb-operator.conf
ExecReload=/bin/kill -HUP $MAINPID
Restart=always
RestartSec=5

# Auto-create and manage /run/sshd lifecycle on startup/reboot
RuntimeDirectory=sshd
RuntimeDirectoryMode=0755

# Hardening, but NOT an empty capability set. The earlier "lock it down fully"
# (CapabilityBoundingSet= empty) was wrong: found on a real host once the port-2222
# conflict stopped hiding it, every connection died pre-auth with
#   sshd: fatal: chroot("/run/sshd"): Operation not permitted [preauth]
# because sshd's privilege separation chroots and switches users. The bounding set is
# the minimum for that model, nothing for binding ports (2222 is unprivileged):
#   CAP_SYS_CHROOT          privsep chroot to /run/sshd
#   CAP_SETUID CAP_SETGID   drop to the privsep user and to `operator` (incl. initgroups)
#   CAP_CHOWN               hand the allocated PTY to the user
#   CAP_KILL                end the session's processes (they run as another uid)
#   CAP_AUDIT_WRITE         audit record on login when sshd is built with audit support
# The PID file is disabled in the sshd config (PidFile none) because ProtectSystem=strict
# makes /run read-only; systemd already tracks the main PID (Type=notify, -D).
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
MemoryDenyWriteExecute=true
ProtectSystem=strict

# Expanded allowed write paths to include the newly mapped runtime space
ReadWritePaths=/run/kb /run/sshd
```

```
CapabilityBoundingSet=CAP_SYS_CHROOT CAP_SETUID CAP_SETGID CAP_CHOWN CAP_KILL CAP_AUDIT_WRITE

[Install]
WantedBy=multi-user.target
```

D.6 kbagents.service

```
[Unit]
Description=Kernel Borderlands Ray Swarm Node Agent (AADS)
# kb-checker is under maintenance and repairs (kept down), so this unit must
# not Require it or starting the swarm would try to start the checker.
After=kbd.service
Requires=kbd.service
DefaultDependencies=no

[Service]
Type=simple
# kb-aads has no compiled build; packaging/bin/kbagents is a launcher that
# runs main.py with the system python3 or /opt/kb/venv and passes --config
# /etc/kb/agents.yaml. It exits 1 with install instructions if ray/grpcio
# are missing, so this unit is installed but NOT enabled by the packages.
# (The old --config /etc/kb/config.yaml --grpc-socket flags matched nothing
# in main.py: the socket comes from agents.yaml's control_plane.grpc_socket.)
ExecStart=/usr/local/bin/kbagents
Restart=always
RestartSec=5

# Full hardening - safe here (unlike kbd/kb-checker above) since this unit
# isn't deployed anywhere yet and its actual runtime need is a UDS client
# connection to kba.sock, nothing privileged. CapabilityBoundingSet= empty
# drops every capability; adjust only if the real kbagents build (once it
# exists - see note above) turns out to need something specific (e.g. a
# Ray-internal requirement not visible from this repo's Python source).
NoNewPrivileges=true
PrivateTmp=true
ProtectHome=true
MemoryDenyWriteExecute=true
ProtectSystem=strict
ReadWritePaths=/run/kb
ReadOnlyPaths=/etc/kb /usr/lib/kb /opt/kb
CapabilityBoundingSet=

[Install]
WantedBy=multi-user.target
```

D.7 kb-tui-session-wrapper.sh

```
#!/bin/sh
# /usr/local/bin/kb-tui-session-wrapper.sh
# ForceCommand target for sshd@kb-operator - reports session start/end into
# kbd's audit log, then runs the unified kbopd binary.
set -u

/usr/local/bin/kbctl ssh session-start --remote-addr "${SSH_CONNECTION:-unknown}"

# FIX: Point directly to the unified binary path managed by your Makefile
/usr/local/bin/kbopd
status=$?

/usr/local/bin/kbctl ssh session-end --remote-addr "${SSH_CONNECTION:-unknown}"

exit "$status"
```

D.8 provision.sh

```
#!/bin/sh
# scripts/setup/provision.sh
#
# Host provisioning for Kernel Borderlands: creates the `kb` group, the
# `operator` system user, /etc/kb, and the SSH host key used by
# sshd@kb-operator.service - none of which any other script in this repo
# creates today. scripts/setup/install.sh explicitly disclaims this exact
# scope ("does NOT... create /run/kb, or touch /etc/kb") - this is the
# script that closes that gap.
#
# Idempotent: safe to re-run. Requires root. Deliberately does NOT touch
# /run/kb (owned by kbd.service's RuntimeDirectory=kb, created/cleaned by
# systemd itself) or /etc/kb/ebpf_policies.json (kb-checker generates a
# default template on first run if missing - see
# kb-checker/src/integrity/mod.rs:21,41).
set -eu

if [ "$(id -u)" -ne 0 ]; then
    echo "provisioning must be run as root" >&2
    exit 1
fi

if ! getent group kb >/dev/null 2>&1; then
    groupadd --system kb
    echo "created group: kb"
else
    echo "group kb already exists, skipping"
fi

if ! getent passwd operator >/dev/null 2>&1; then
    # Shell is /bin/sh, not nologin: sshd runs a ForceCommand as "$SHELL -c <cmd>", which a
    # nologin shell refuses ("This account is currently not available"). The account stays
    # locked down by the operator sshd config: key-only, ForceCommand wrapper, no forwarding,
    # no password.
```

```
    # Home is / (not /home/operator): the account has no home, sshd chdirs to it on every login
    # and warns "Could not chdir to home directory" because the unit hides /home (ProtectHome).
    useradd --system --no-create-home --home-dir / --shell /bin/sh --gid kb operator
    # useradd leaves the shadow password as "!" (locked). With UsePAM no (our standalone sshd
    # config) OpenSSH refuses a locked account even for public-key logins ("User operator not
    # allowed because account is locked"). "*" = no usable password, but not locked.
    usermod -p '*' operator
    echo "created user: operator (member of kb)"
else
    echo "user operator already exists, skipping"
fi

mkdir -p /etc/kb
mkdir -p /etc/kb/ssh
chown root:kb /etc/kb
chmod 0750 /etc/kb

HOST_KEY=/etc/kb/ssh/ssh_host_ed25519_key
if [ ! -f "$HOST_KEY" ]; then
    ssh-keygen -t ed25519 -f "$HOST_KEY" -N ""
    echo "generated SSH host key: $HOST_KEY"
else
    echo "SSH host key already exists, skipping: $HOST_KEY"
fi

AUTH_KEYS=/etc/kb/authorized_keys
if [ ! -f "$AUTH_KEYS" ]; then
    touch "$AUTH_KEYS"
    chown operator:kb "$AUTH_KEYS"
    chmod 0600 "$AUTH_KEYS"
    echo "created empty $AUTH_KEYS - add operator public keys here"
else
    echo "$AUTH_KEYS already exists, skipping"
fi

echo "provisioning complete"
```

APPENDIX E

GLOSSARY

BTF
BPF Type Format: kernel type information that lets compiled eBPF relocate itself.

CO-RE
Compile Once, Run Everywhere: eBPF built once and adapted to each kernel via BTF.

BPF LSM
A Linux Security Module whose hooks are eBPF programs; the only mechanism here that can *block*.

Zone
`SAFE`, `SUSPICIOUS`, `BORDERLANDS`: **a process's score band.**

Behavior state
`SAFE`, `OBSERVED`, `SUSPICIOUS`, `BORDERLANDS`, `COMPROMISED`: **states in the rules engine.**

Containment level
`CGROUP`, `SECCOMP`, `NAMESPACE`, `TERMINATE` **(TUI also offers** `none` **to release).**

CWP
Critical Workload Protection: an optional, signable registry of programs to guard.

UDS
Unix domain socket.

Drop-in
A systemd override file in `/etc/systemd/system/UNIT.d/` **that survives package upgrades.**

Conffile / backup
A config file the package manager will not overwrite without asking.

ForceCommand
An `sshd` **option that runs a fixed program no matter what the client requests.**

MCP
Model Context Protocol: how AI assistants query `mcp-server`.

APPENDIX F

PACKAGE FILE MANIFEST

Path	Mode	Purpose
/usr/bin/kbd	0755	Control plane daemon
/usr/bin/kbd_sensor	0755	eBPF loader and bridge
/usr/bin/kbctl	0755	Operator CLI
/usr/bin/kbopd	0755	TUI and embedded dashboard
/usr/bin/mcp-server	0755	MCP server
/usr/bin/kb-checker	0755	Watchdog (held down)
/usr/bin/kbagents	0755	Swarm launcher
/usr/bin/kb-tui-session-wrapper.sh	0755	SSH ForceCommand target
/usr/lib/kb/provision.sh	0755	Host provisioning (group, operator account, host key)
/usr/lib/kb/migrate-legacy.sh	0755	Install/upgrade step: API token, operator unlock, legacy-install repair, kb-checker autostart removal
/etc/kb/api.env (generated)	0640 root:kb	Dashboard API token
/usr/lib/kb/aads/**	0644	kb-aads sources and the containment checkpoint (with marl/checkpoints/SHA256SUMS)
/usr/lib/systemd/system/*.service (6)	0644	Unit files
/etc/kb/policy.yaml, rules.yaml, workloads.yaml, agents.yaml	0640	Default configs (protected)
/etc/kb/sshd_kb-operator.conf	0644	Operator sshd config (protected)
/usr/share/doc/kernel-borderlands/*	0644	LICENSE, README